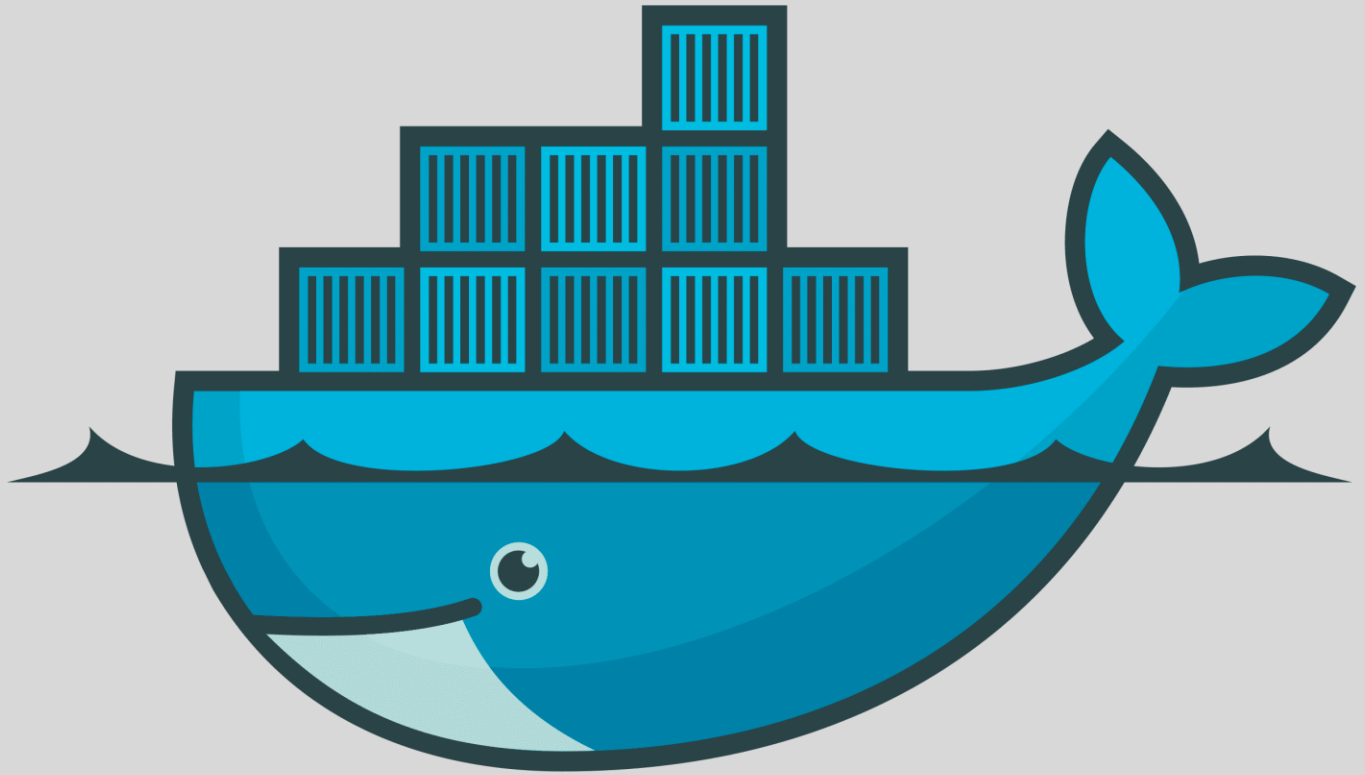
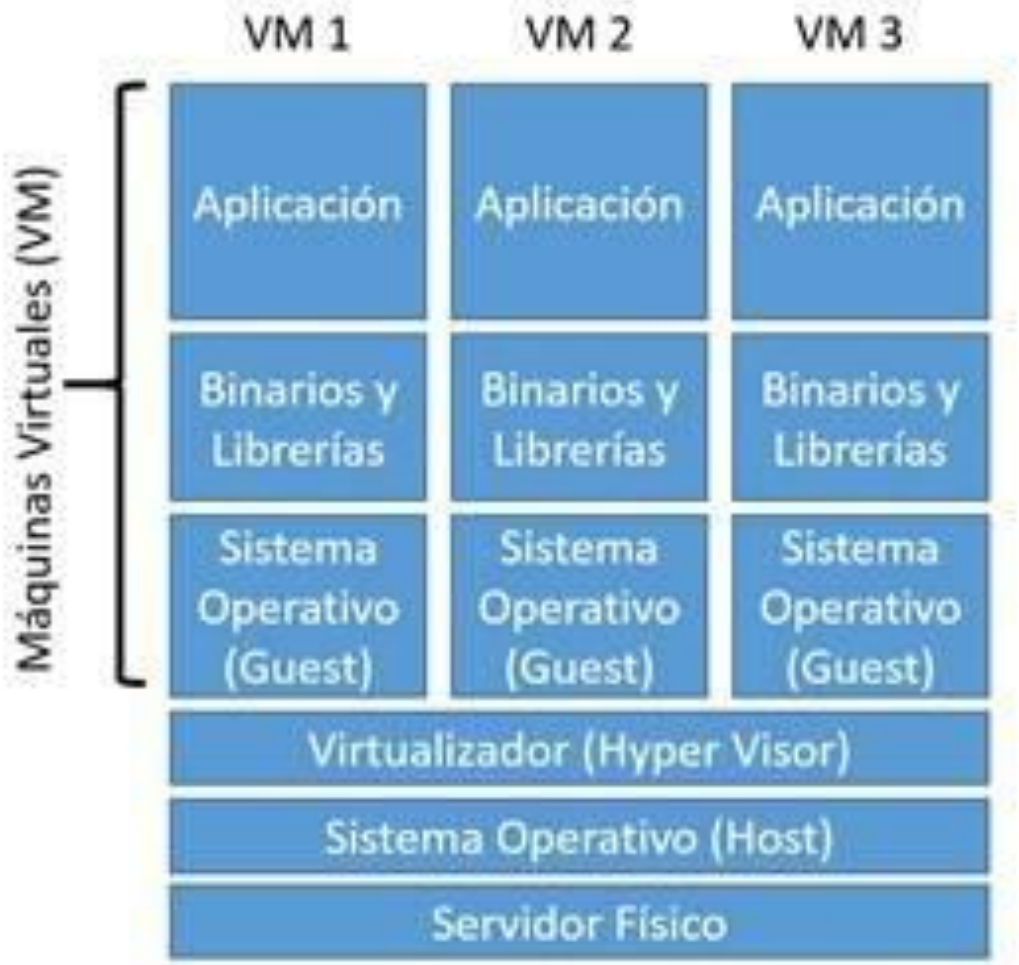
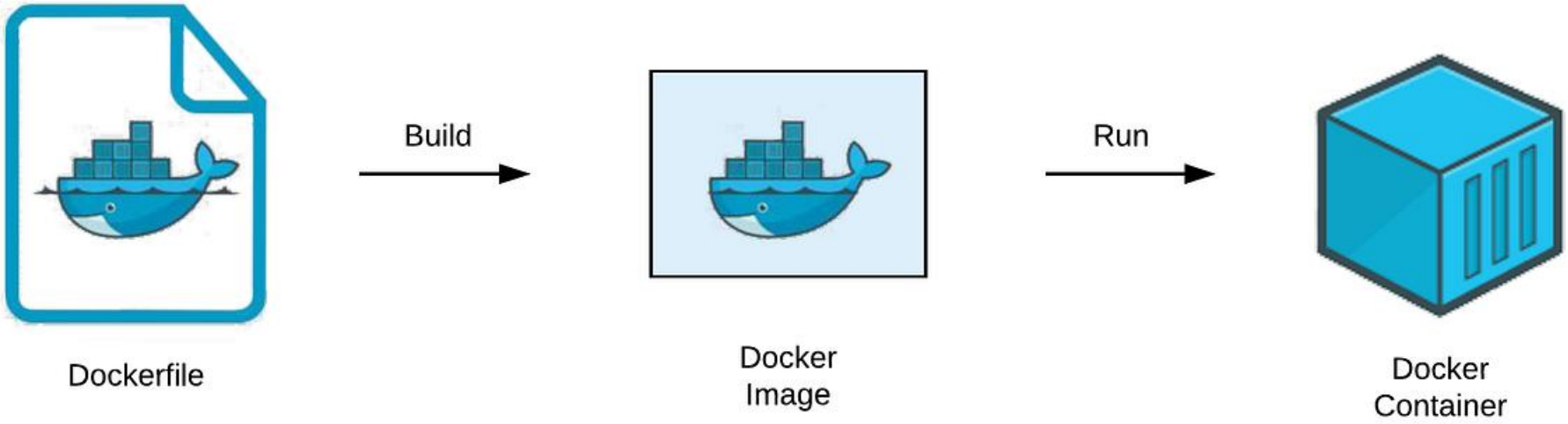
DOCKER



docker


VM 1
VM 2
VM 3
Máquinas Virtuales (VM)
Aplicación
Aplicación
Aplicación
Binarios y Librerías
Binarios y Librerías
Binarios y Librerías
Sistema Operativo (Guest)
Sistema Operativo (Guest)
Sistema Operativo (Guest)
Virtualizador (Hyper Visor)
Sistema Operativo (Host)
Servidor Físico

App 1
App 2
App 3
Contenedores
Aplicación
Aplicación
Aplicación
Sistema Operativo (Guest), Binarios y Librerías
Sistema Operativo (Host)
Servidor Físico


Dockerfile
Build
Docker
Image
Run
Docker
Container

Builder main commands

command	description
`FROM` *image*|`scratch`	base image for the build
`MAINTAINER` *email*	name of the mainainer (metadata)
`COPY` *path dst*	copy *path* from the context into the container at location *dst*
`ADD` *src dst*	same as `COPY` but untar archives and accepts http urls
`RUN` *args...*	run an arbitrary command inside the container
`USER` *name*	set the default username
`WORKDIR` *path*	set the default working directory
`CMD` *args...*	set the default command
`ENV` *name value*	set an environment variable

Una vez terminada la construcción de la imagen, tendremos que hacer, ahora, uso de otros comandos que nos permitirán crear nuestro contenedor y gestionar el mismo. En concreto:

- Para **crear un contenedor** a partir de una imagen:

```
docker run -it --name <container-name> -p <host-port>:<container-port> -d <image-name>
```

- Para ver los **contenedores existentes** que se encuentren levantados:

```
docker ps
```

- Para **ver todos los contenedores**, independientemente de su estado:

```
docker ps -a
```

- Para **acceder al contenedor** o ejecutar comandos en el mismo:

```
docker exec -it <container-name|id> <command>
```

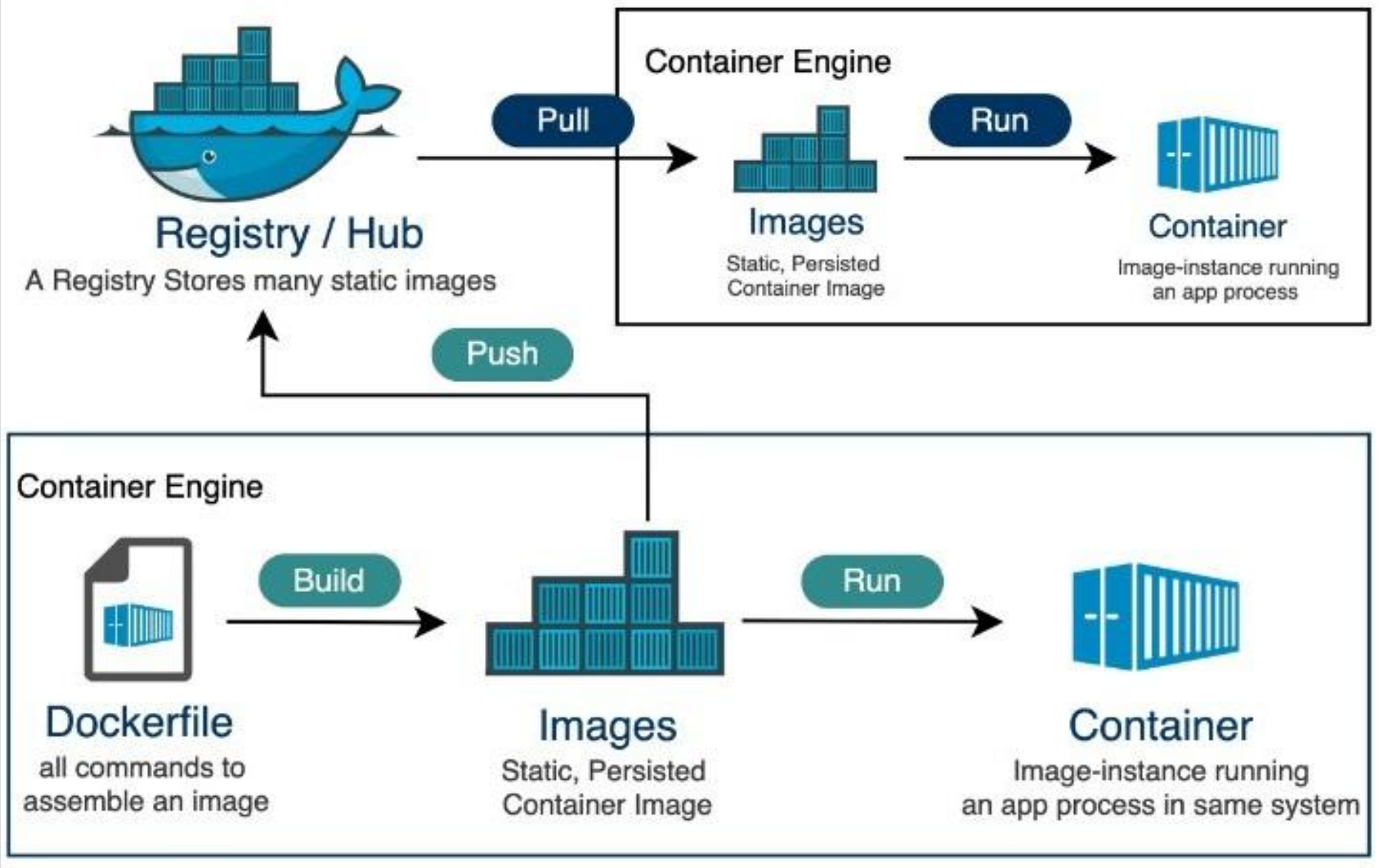
- Por ejemplo, para acceder por terminal, o línea de comandos, podríamos utilizar el comando:

```
docker exec -it <container-name> /bin/bash
```

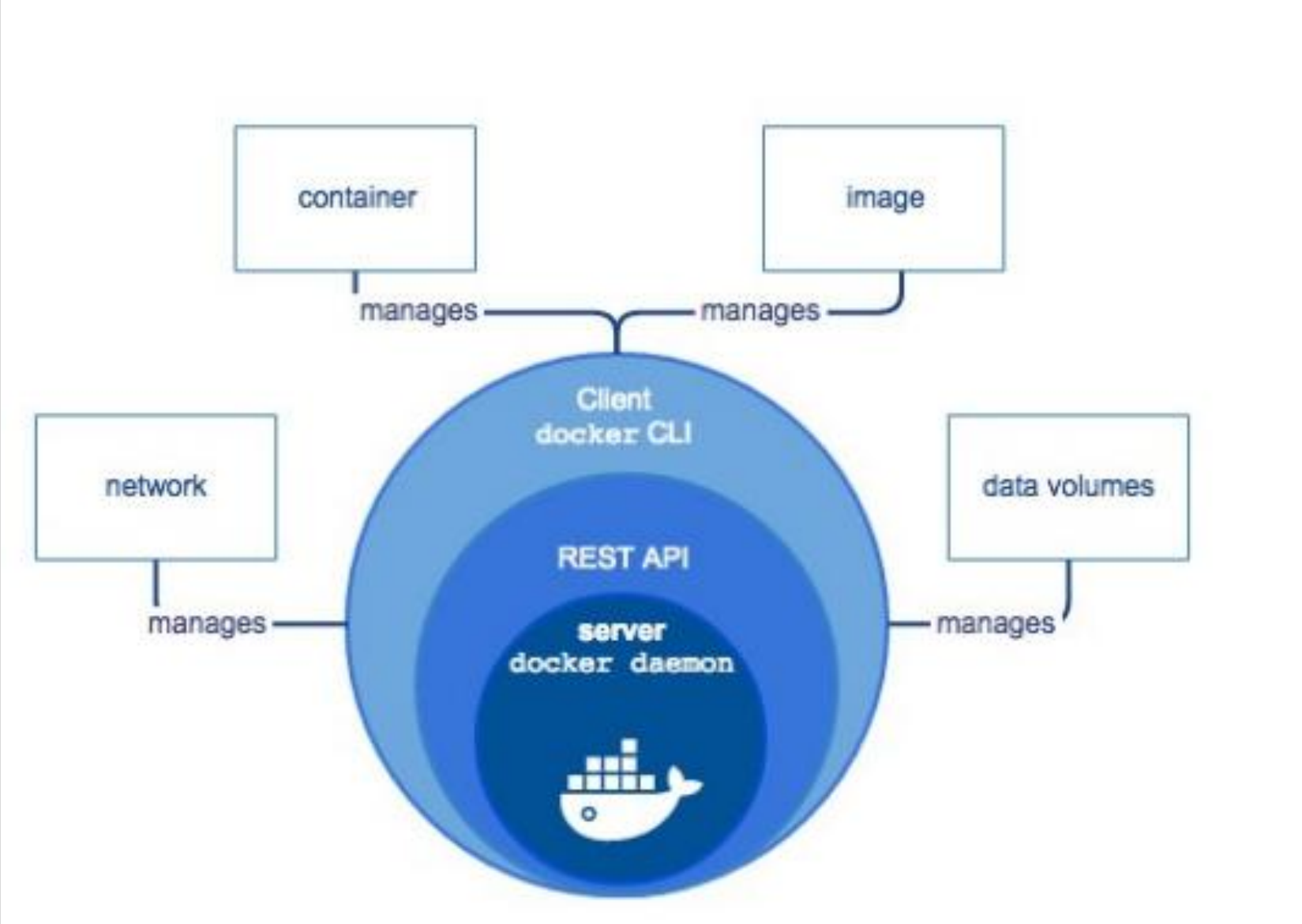
> `/bin/bash` indica que los scripts o comandos ejecutados serán interpretados con Bash, el *shell* (o intérprete de comandos) de Linux. Lo veremos en más ocasiones más adelante. [19]

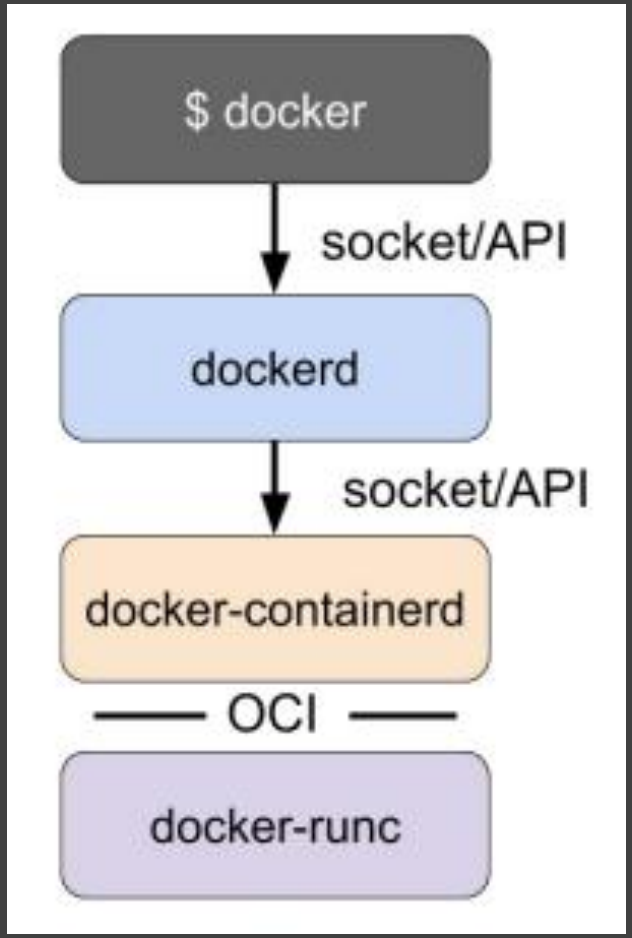
- Para comprobar el **Healthcheck**:

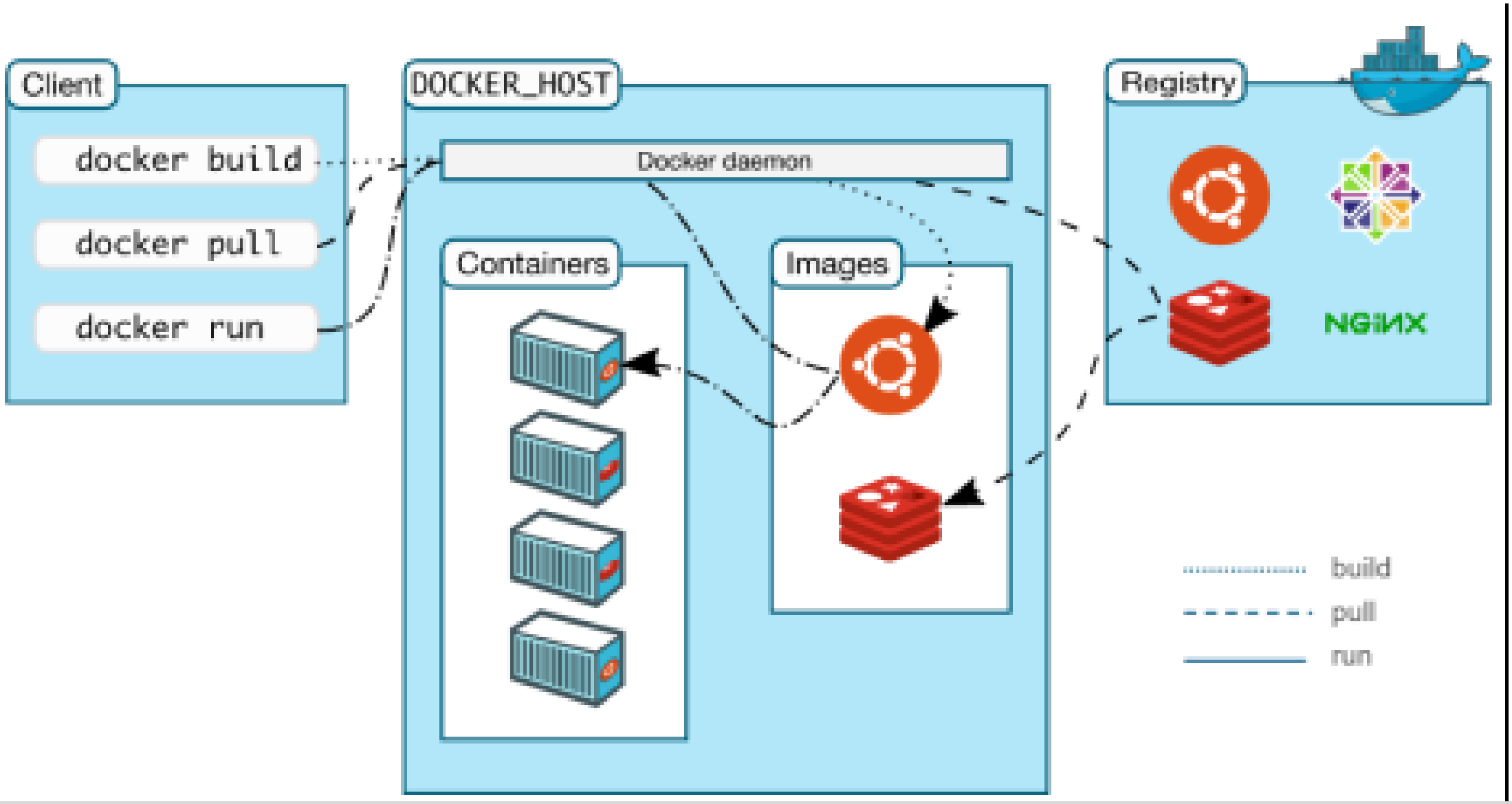
```
docker inspect --format='{{json .State.Health}}' <container-name>
```


Registry / Hub
A Registry Stores many static images
Pull
Container Engine
Images
Static, Persisted
Container Image
Run
Container
Image-instance running
an app process
Push
Container Engine
Dockerfile
all commands to
assemble an image
Build
Images
Static, Persisted
Container Image
Run
Container
Image-instance running
an app process in same system

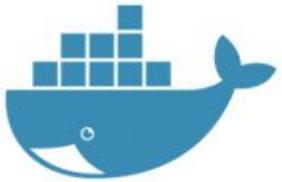
LABORATORIO


container
image
manages
manages
Client
docker CLI
network
data volumes
REST API
manages
server
docker daemon
manages

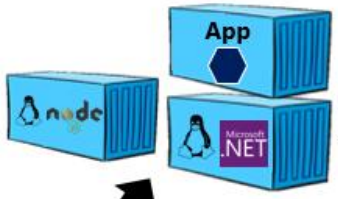

$ docker
socket/API
dockerd
socket/API
docker-containerd
OCI
docker-runc


Client
docker build
docker pull
docker run
DOCKER_HOST
Docker daemon
Containers
Images
Registry
NGINX
build
pull
run

Basic taxonomy in Docker



A **Registry**
Stores many
static images



Images
Static, persisted container image

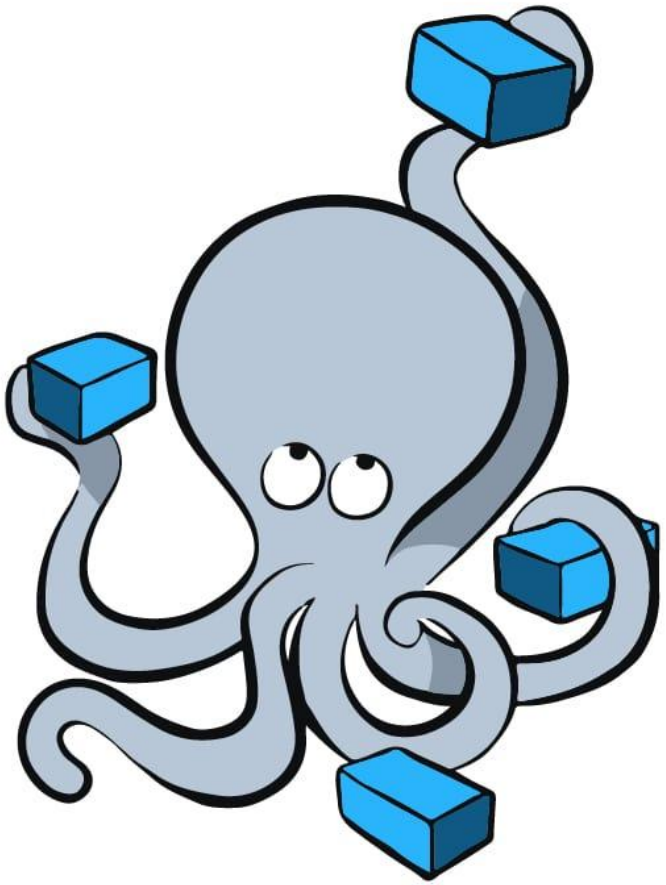
Container
Image-instance running
an app process (service/web)

On-premises
('n' private organizations)

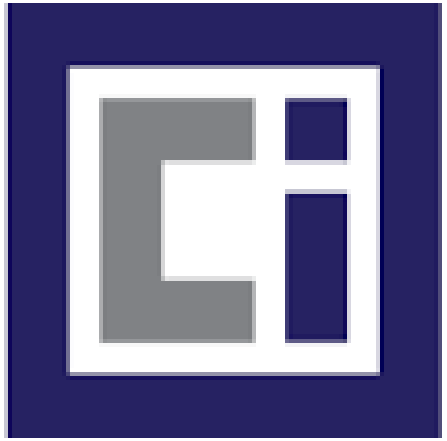
- **Hosted Docker Registry**
- **Docker Trusted Registry on-prem.**

Public Cloud
(specific vendors)

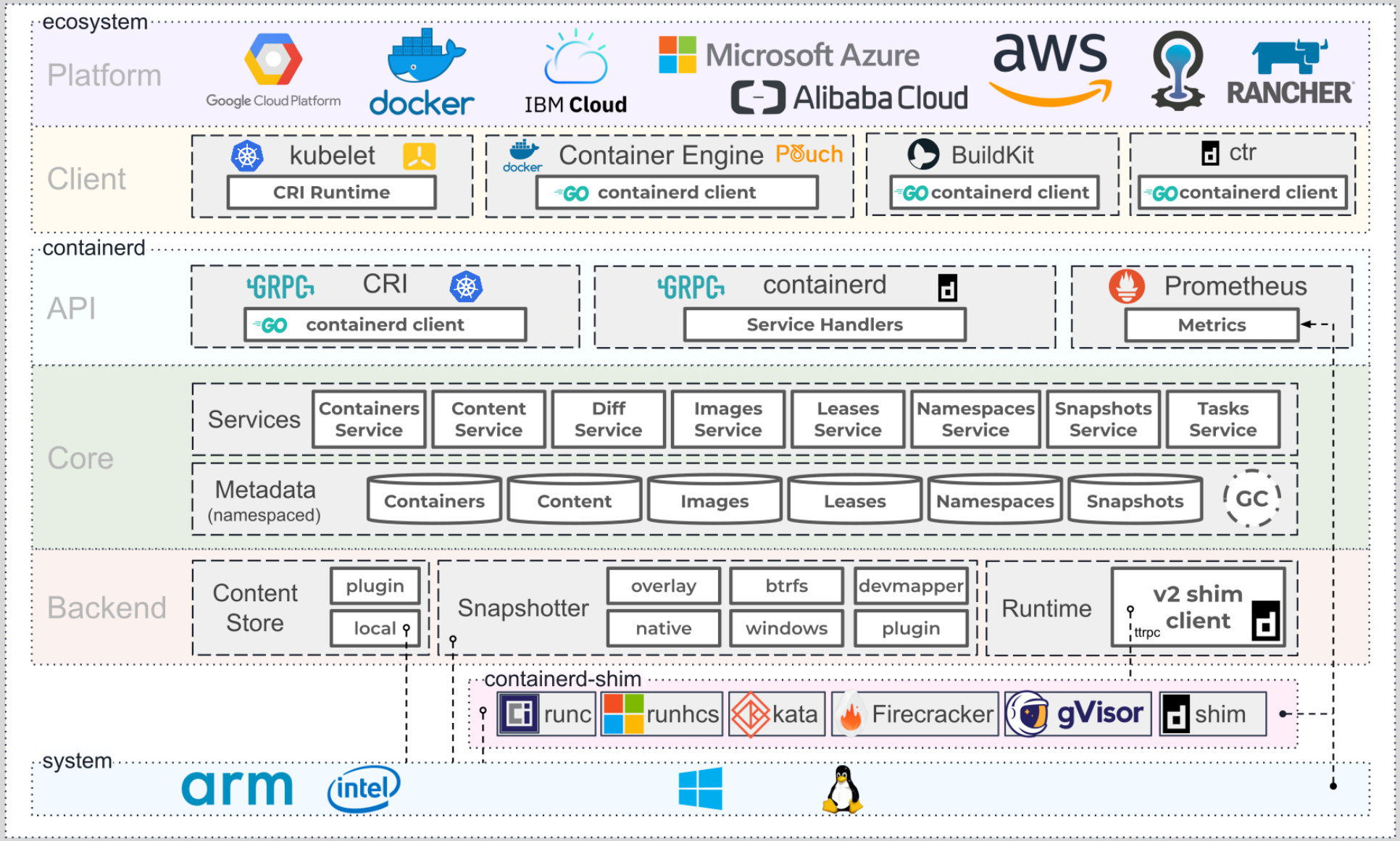
- **Docker Hub Registry**
- Docker Trusted Registry on-cloud
- **Azure Container Registry**
- AWS Container Registry
- Google Container Registry
- Quay Registry

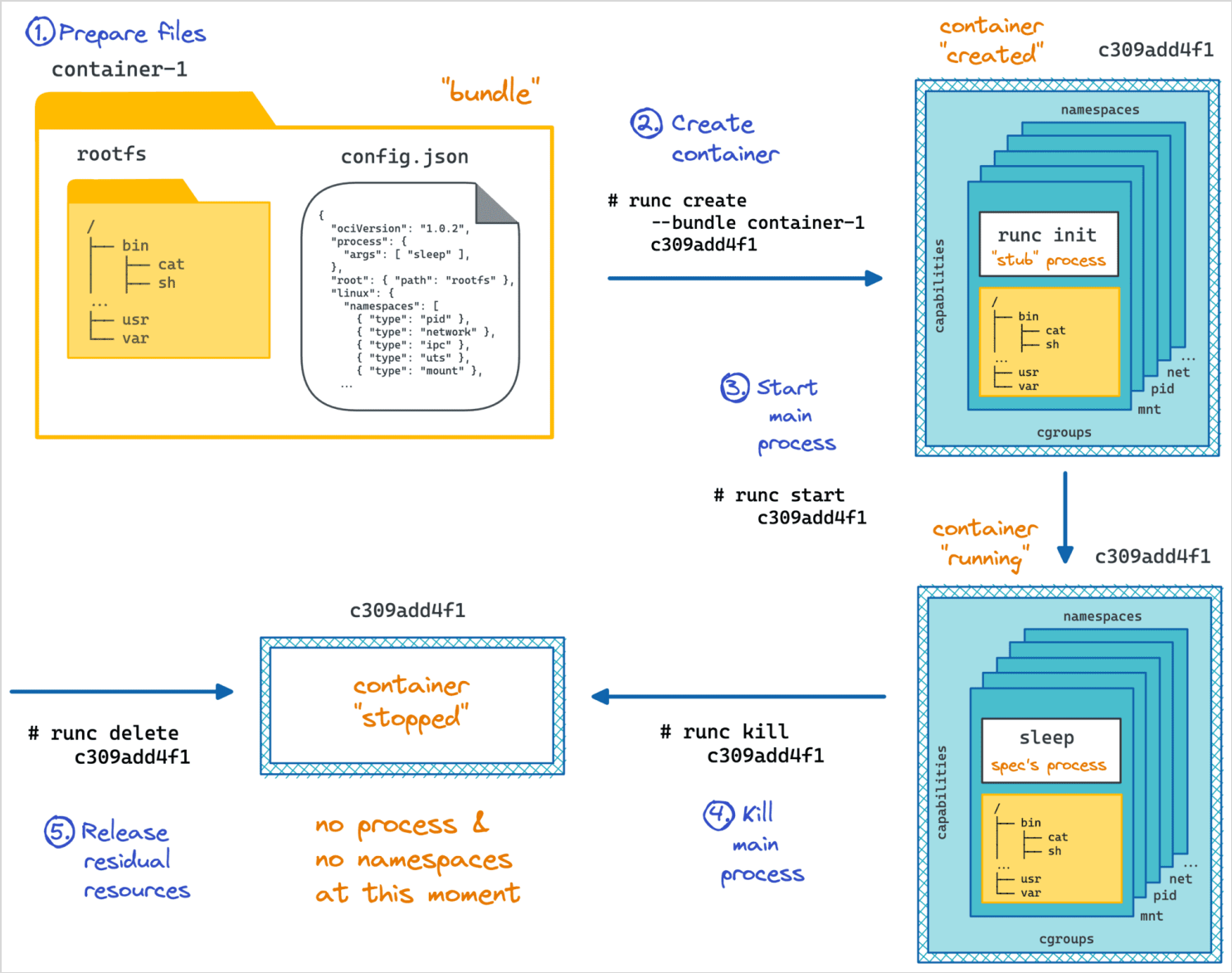


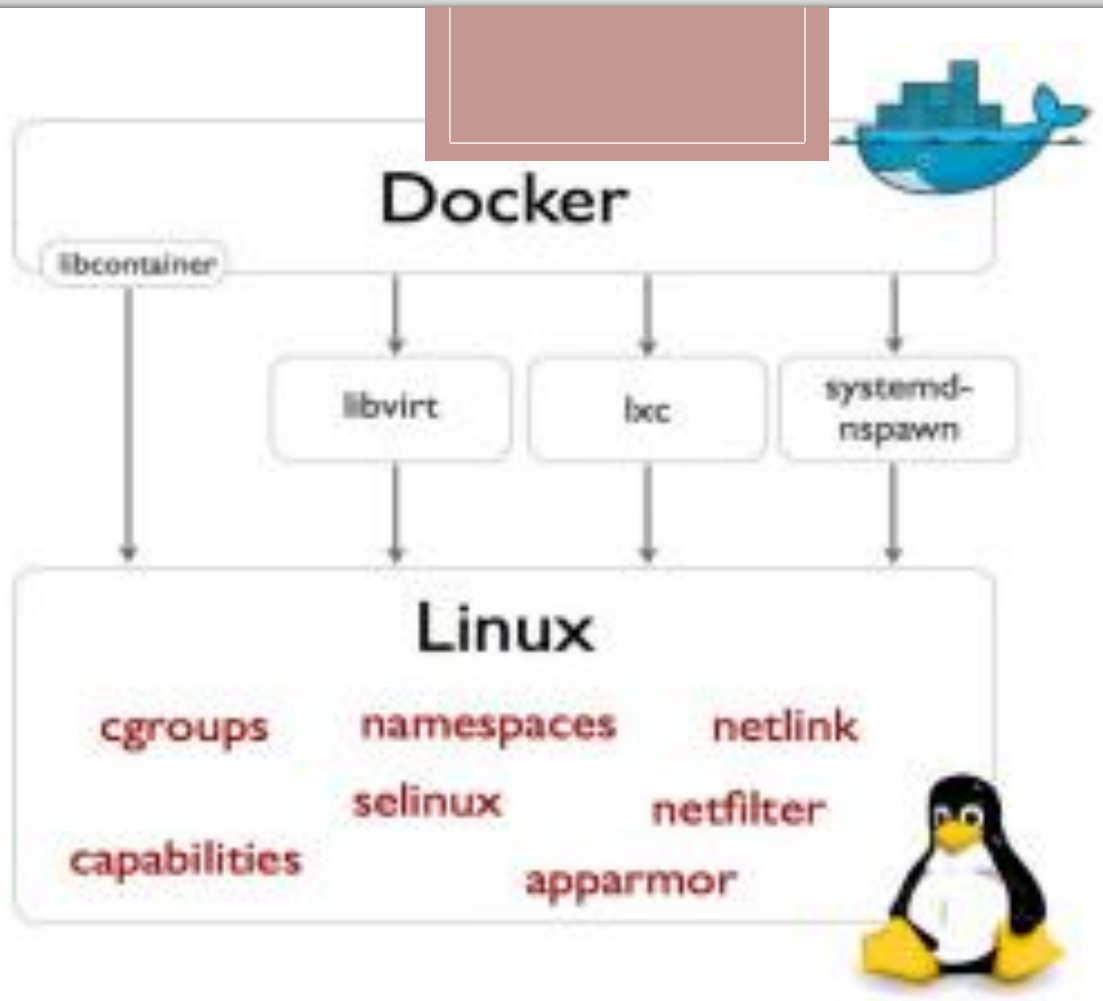
docker
Compose



OPEN CONTAINER INITIATIVE

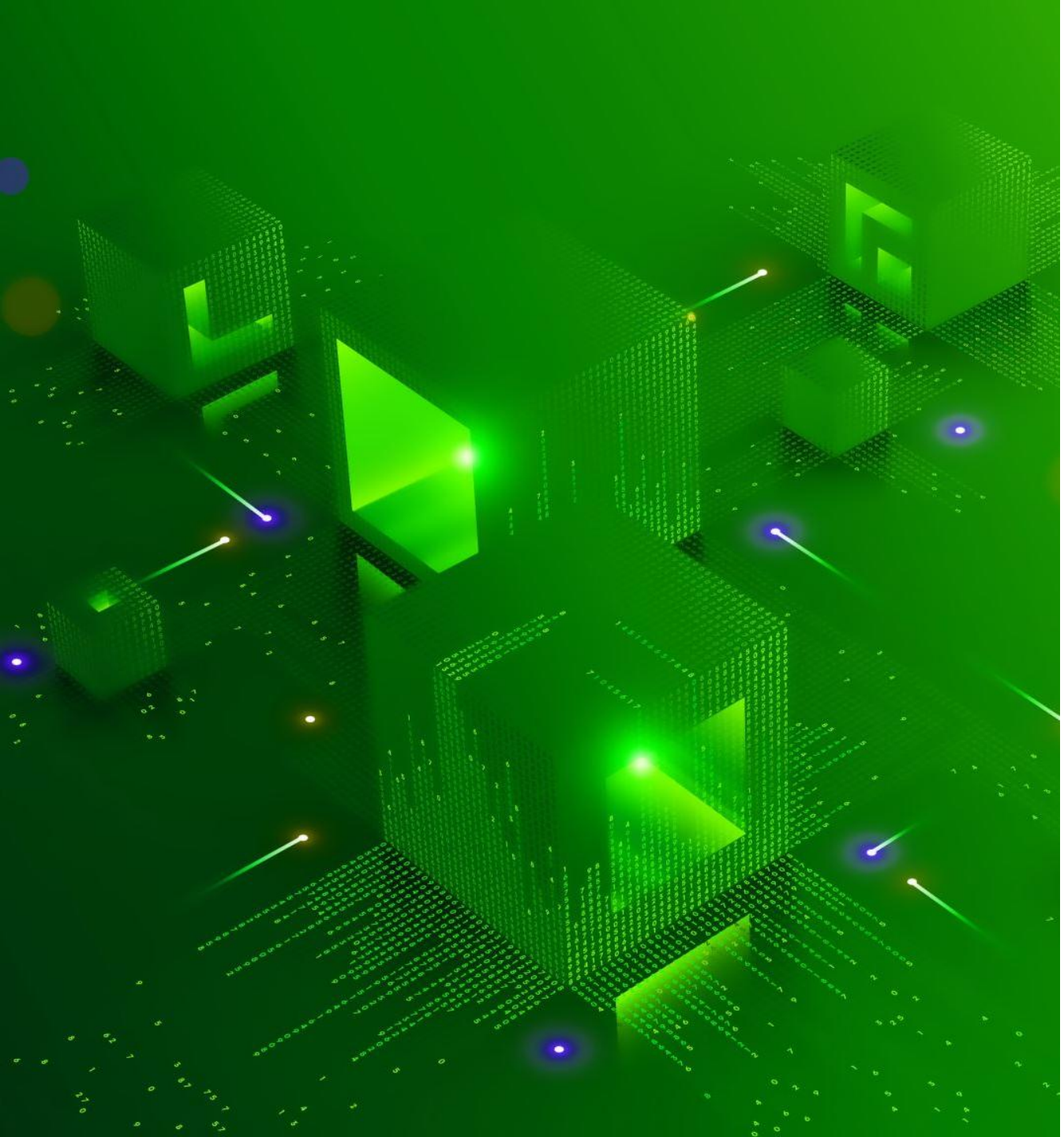

ecosystem
Platform
Google Cloud Platform
docker
IBM Cloud
Microsoft Azure
Alibaba Cloud
aws
RANCHER
Client
kubelet
CRI Runtime
Container Engine
containerd client
BuildKit
containerd client
ctr
containerd client
containerd
API
GRPC
CRI
containerd client
GRPC
containerd
Service Handlers
Prometheus
Metrics
Core
Services
Containers Service
Content Service
Diff Service
Images Service
Leases Service
Namespaces Service
Snapshots Service
Tasks Service
Metadata
(namespaced)
Containers
Content
Images
Leases
Namespaces
Snapshots
GC
Backend
Content Store
plugin
local
Snapshotter
overlay
btrfs
devmapper
native
windows
plugin
Runtime
v2 shim client
ttrpc
containerd-shim
runc
runhcs
kata
Firecracker
gVisor
shim
system
arm
intel


1. Prepare files
container-1
"bundle"
rootfs
/
bin
cat
sh
...
usr
var
config.json
{
"ociVersion": "1.0.2",
"process": {
"args": ["sleep"],
},
"root": { "path": "rootfs" },
"linux": {
"namespaces": [
{ "type": "pid" },
{ "type": "network" },
{ "type": "ipc" },
{ "type": "uts" },
{ "type": "mount" },
...
2. Create container
runc create
--bundle container-1
c309add4f1
container "created"
c309add4f1
namespaces
capabilities
runc init
"stub" process
/
bin
cat
sh
...
usr
var
net
pid
mnt
cgroups
3. Start main process
runc start
c309add4f1
container "running"
c309add4f1
namespaces
capabilities
sleep
spec's process
/
bin
cat
sh
...
usr
var
net
pid
mnt
cgroups
runc kill
c309add4f1
4. Kill main process
c309add4f1
container "stopped"
no process &
no namespaces
at this moment
runc delete
c309add4f1
5. Release residual resources


Docker
libcontainer
libvirt
lxc
systemd-
nspawn
Linux
cgroups
namespaces
netlink
selinux
netfilter
capabilities
apparmor

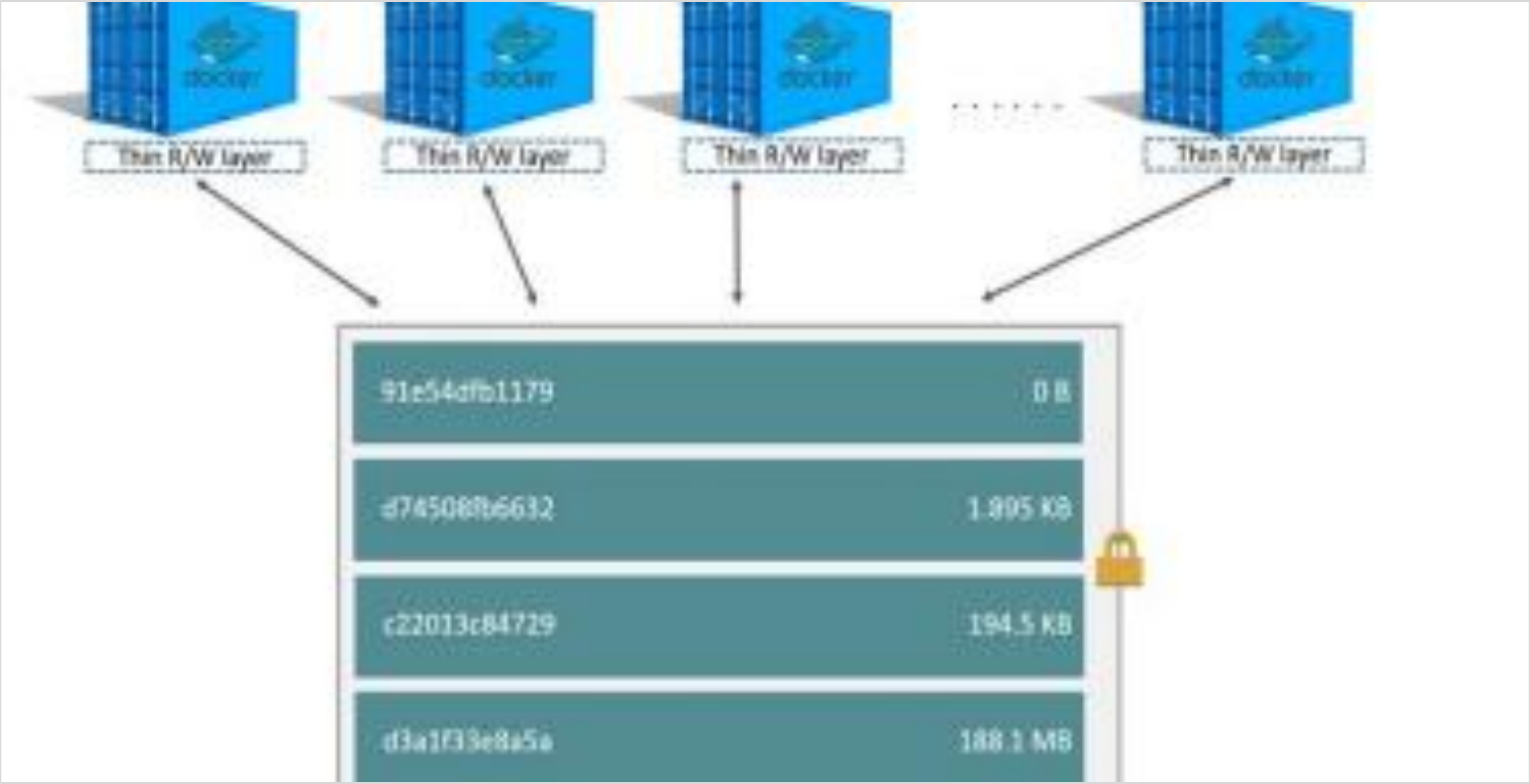
NAMESPACE

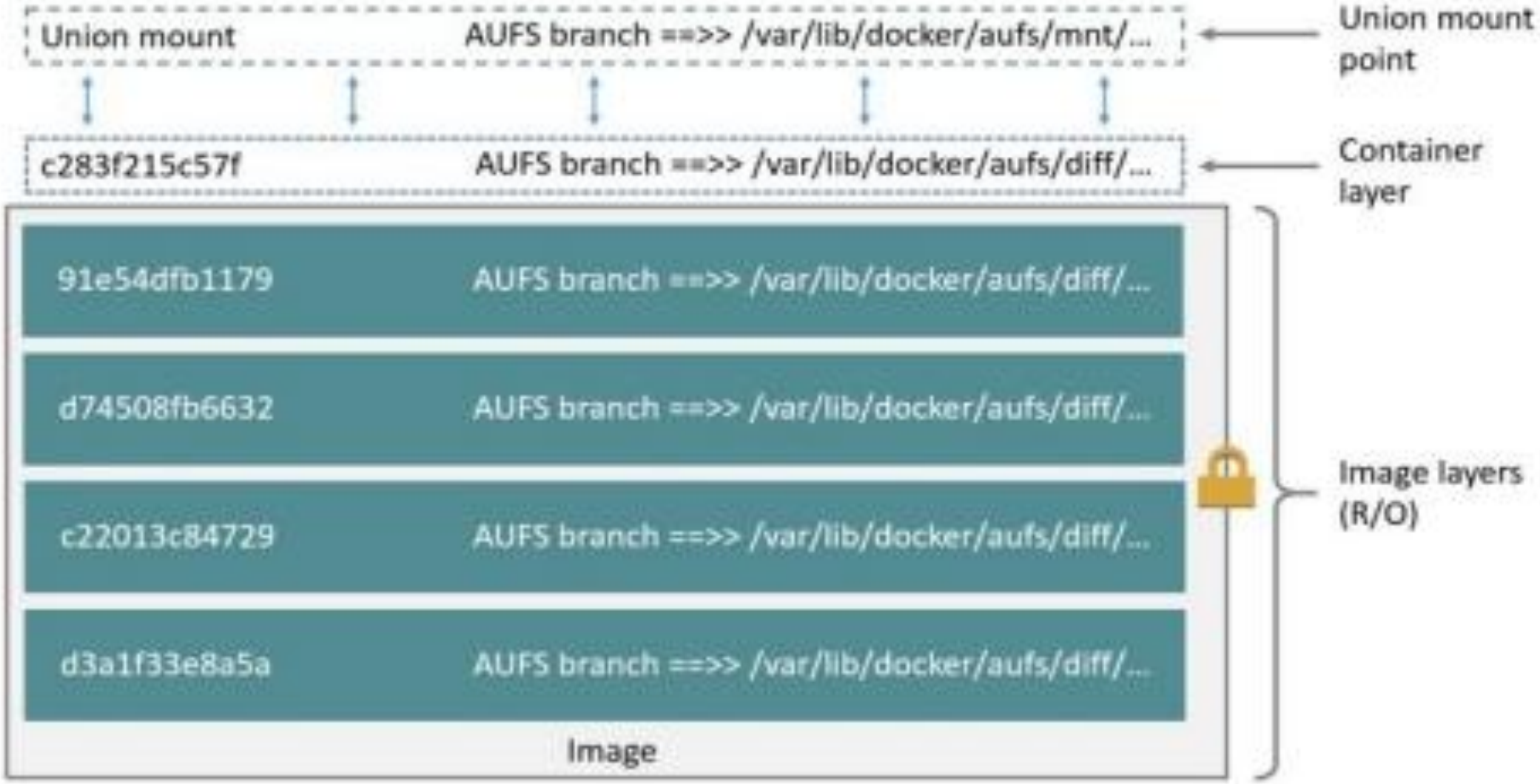
- Un espacio de nombres, o namespace, es una tecnología del kernel de Linux que provee espacios de trabajo separados y una capa de aislamiento, haciendo que cada contenedor solo pueda ver los recursos pertenecientes a su namespace. De esta forma, Docker (runc), al crear un contenedor, crea también un conjunto de namespaces para dicho contenedor.

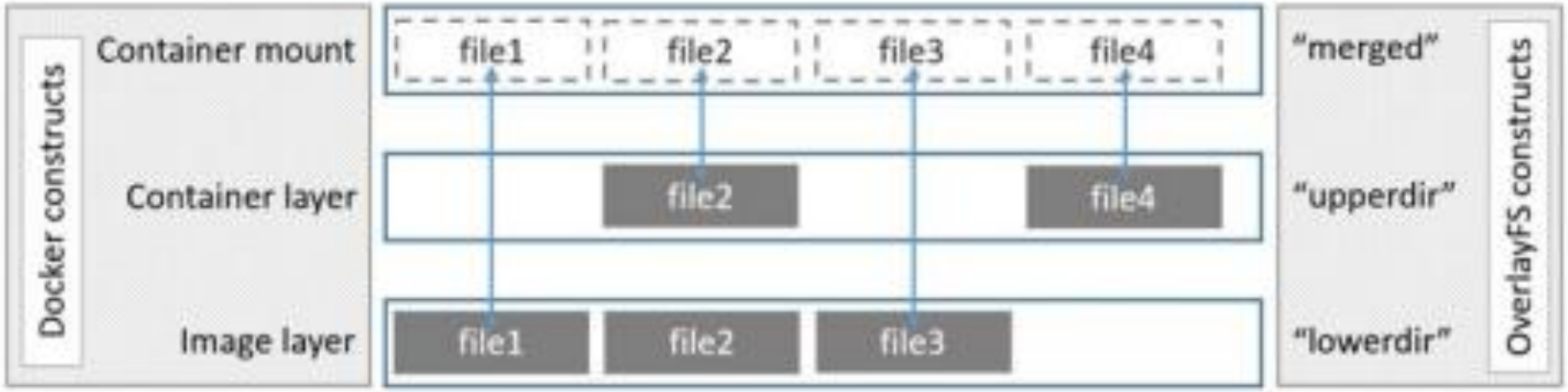


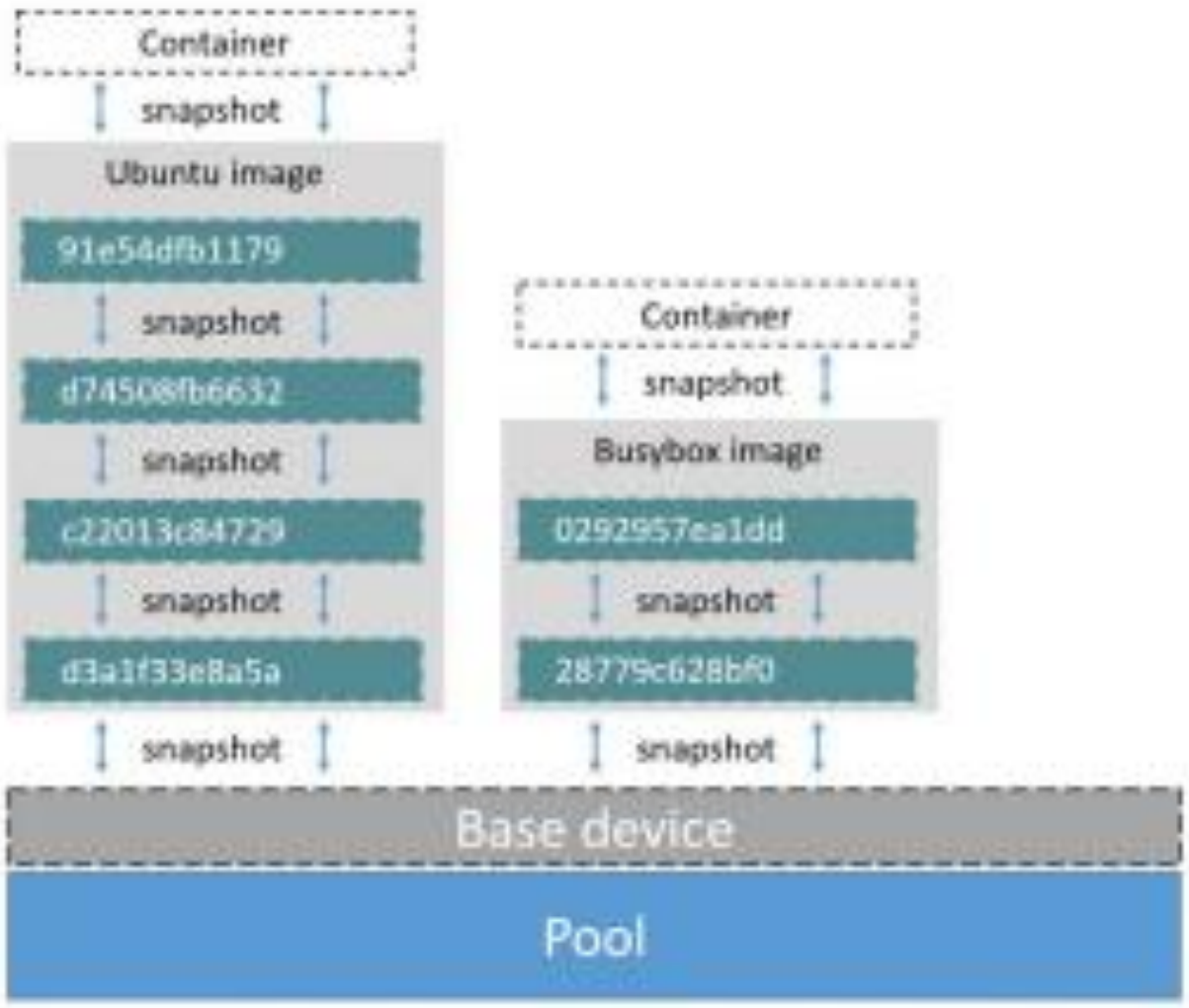
CGROUPS

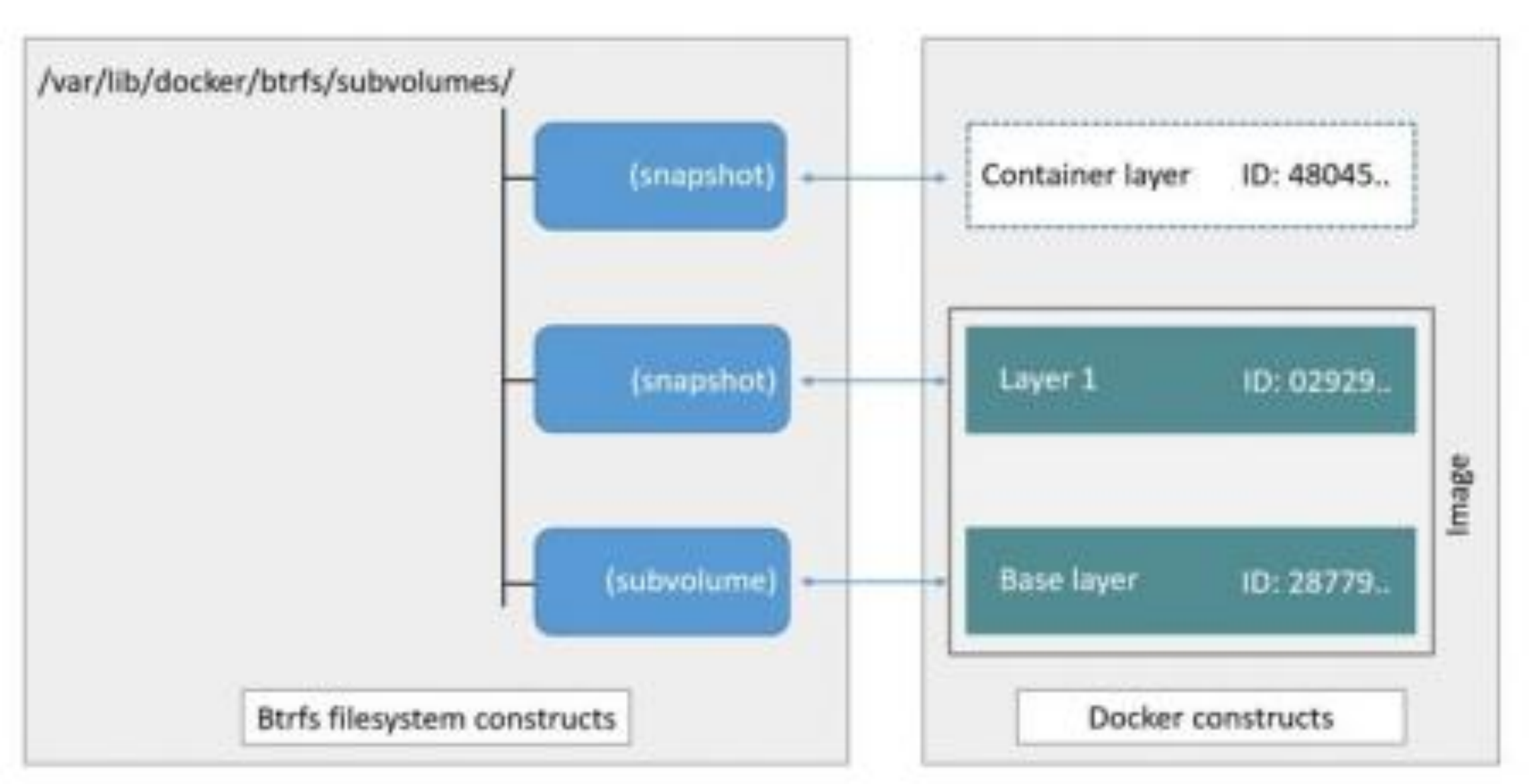
- Otra tecnología del kernel de Linux es Control Groups (cgroups) que permite establecer limitaciones en el uso de un conjunto de recursos para un proceso, compartiendo los recursos entre contenedores y estableciendo distintos tipos de límites. También permite hacer accounting (contabilizar el uso de recursos), /dev (cierto control de acceso a los ficheros de dispositivos), congelar grupos de procesos y otras cosas.
- Los cgroups se activan globalmente (es decir, están habilitados para todos los procesos, o para ninguno) y son jerárquicos, con un nodo raíz por cada tipo de recurso.

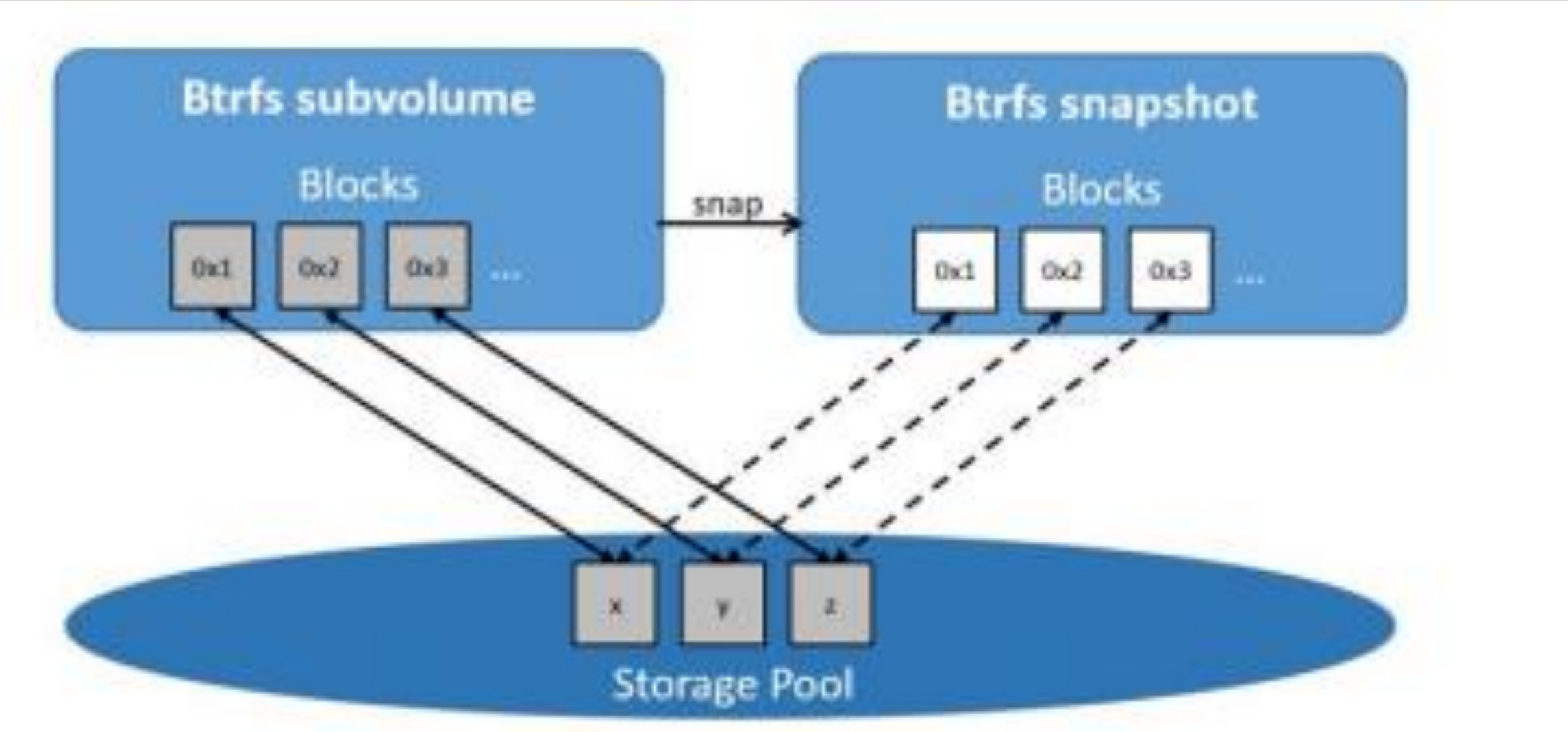

Thin R/W layer
Thin R/W layer
Thin R/W layer
Thin R/W layer
91e54dfb1179
0 B
d74508fb6632
1.895 KB
c22013c84729
194.5 KB
d3a1f33e8a5a
188.1 MB

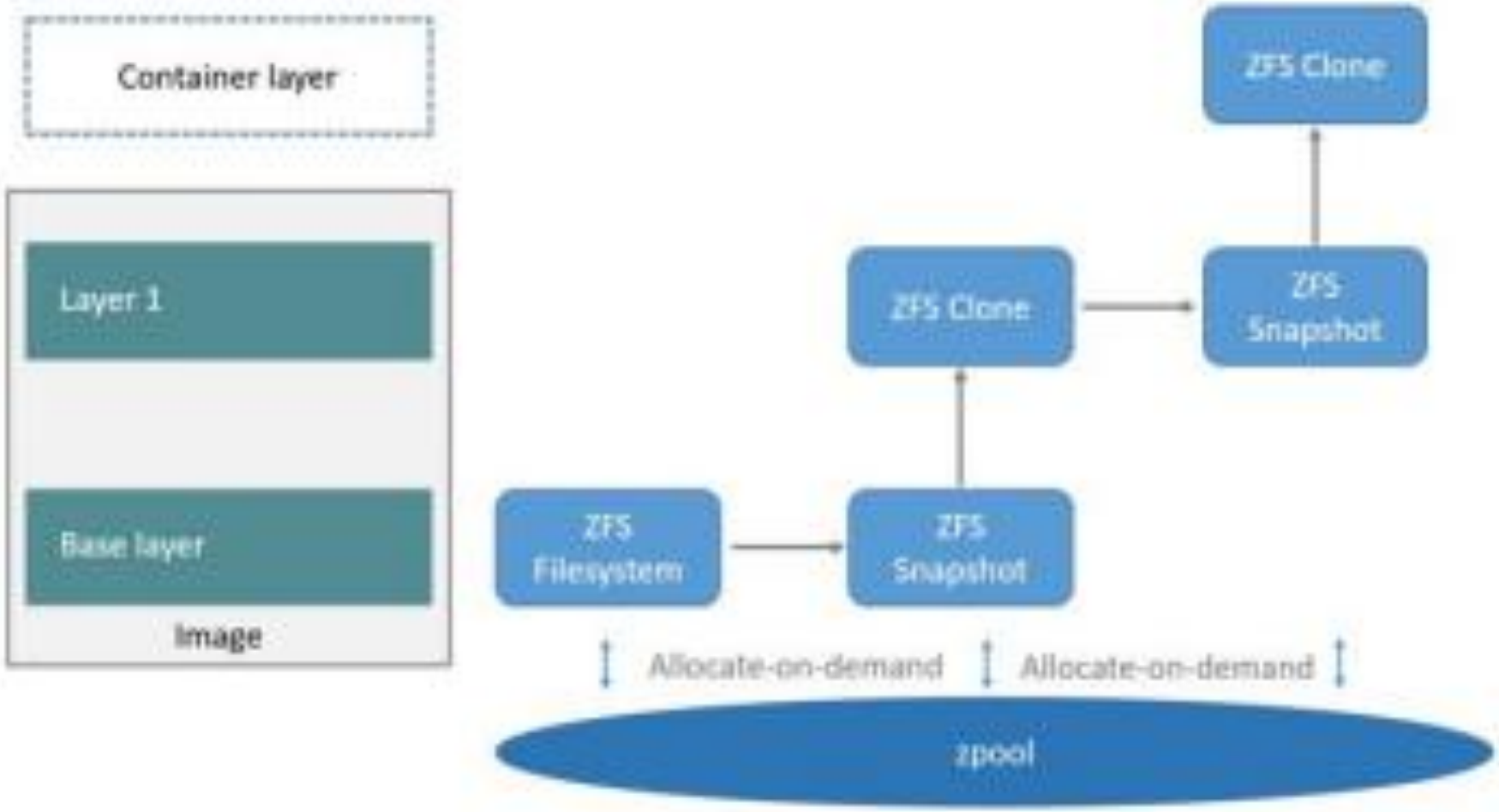

Union mount
AUFS branch ==>> /var/lib/docker/aufs/mnt/...
Union mount point
c283f215c57f
AUFS branch ==>> /var/lib/docker/aufs/diff/...
Container layer
91e54dfb1179
AUFS branch ==>> /var/lib/docker/aufs/diff/...
d74508fb6632
AUFS branch ==>> /var/lib/docker/aufs/diff/...
c22013c84729
AUFS branch ==>> /var/lib/docker/aufs/diff/...
d3a1f33e8a5a
AUFS branch ==>> /var/lib/docker/aufs/diff/...
Image
Image layers (R/O)

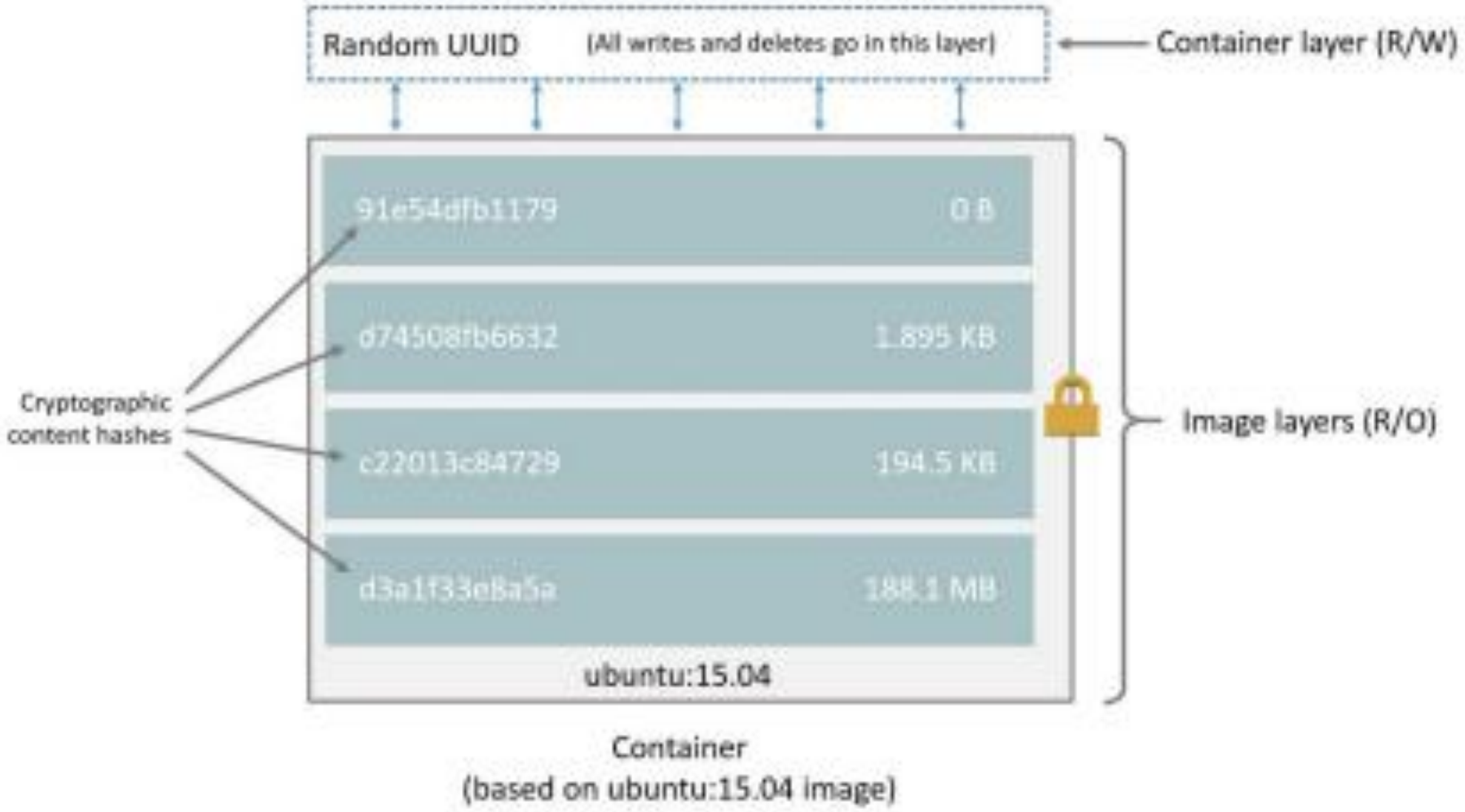

Docker constructs
Container mount
Container layer
Image layer
file1
file2
file3
file4
file2
file4
file1
file2
file3
"merged"
"upperdir"
"lowerdir"
OverlayFS constructs

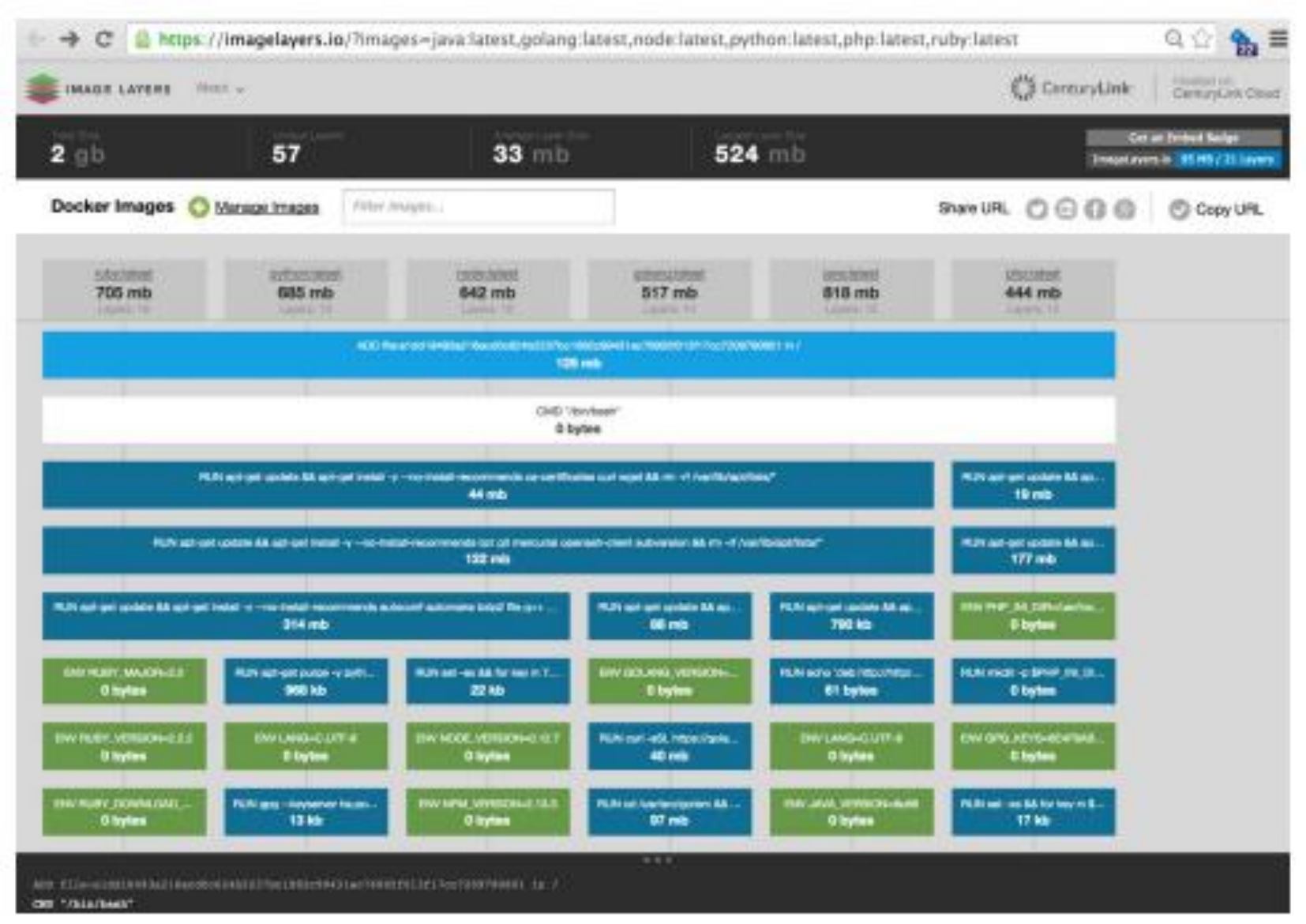

Container
snapshot
Ubuntu image
91e54dfb1179
snapshot
d74508fb6632
snapshot
c22013c84729
snapshot
d3a1f33e8a5a
snapshot
Container
snapshot
Busybox image
0292957ea1dd
snapshot
28779c628bf0
snapshot
Base device
Pool


/var/lib/docker/btrfs/subvolumes/
(snapshot)
(snapshot)
(subvolume)
Btrfs filesystem constructs
Container layer
ID: 48045..
Layer 1
ID: 02929..
Base layer
ID: 28779..
Image
Docker constructs


Btrfs subvolume
Blocks
0x1
0x2
0x3
...
snap
Btrfs snapshot
Blocks
0x1
0x2
0x3
...
x
y
z
Storage Pool

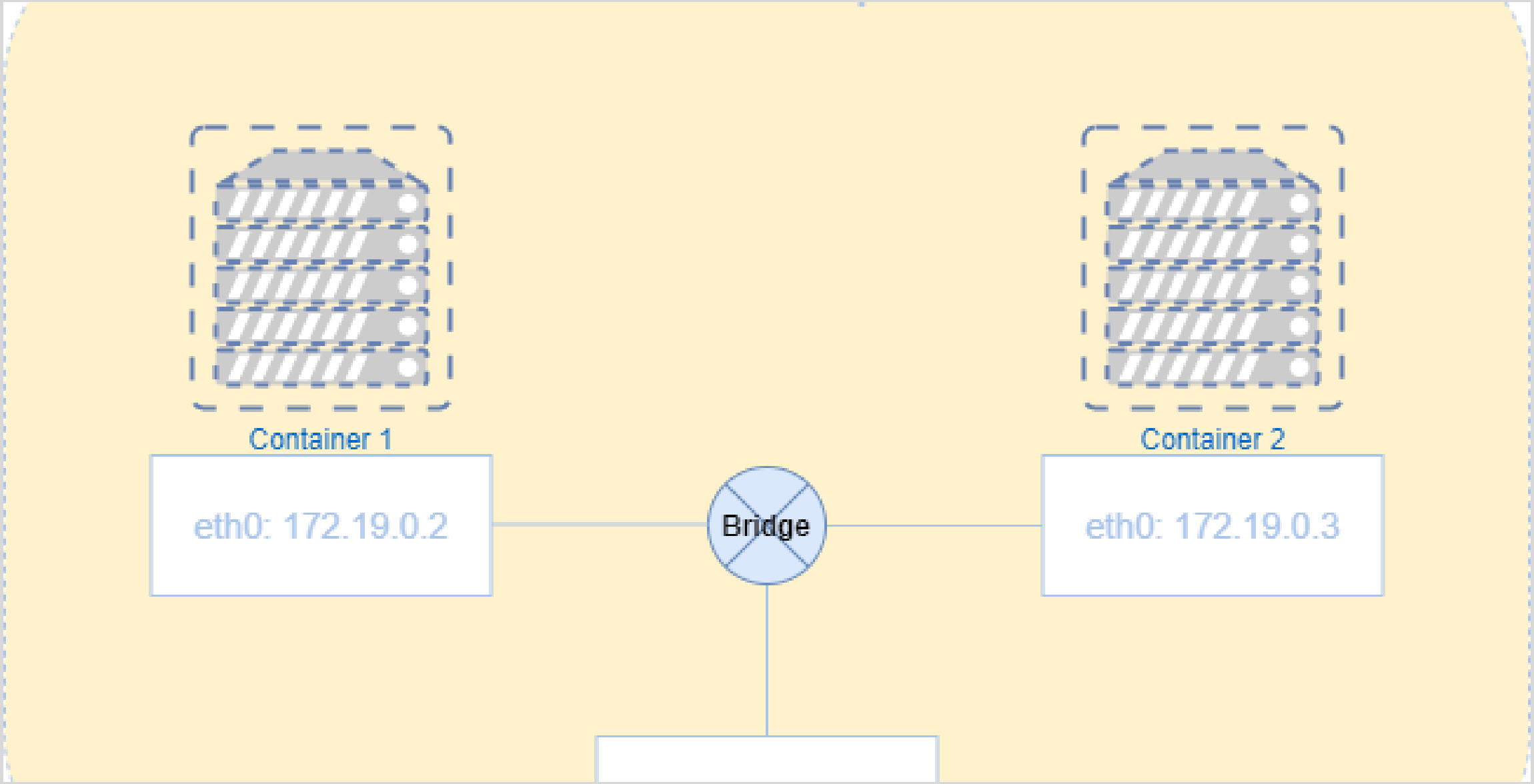

Container layer
Layer 1
Base layer
Image
ZFS Clone
ZFS Clone
ZFS Snapshot
ZFS Filesystem
ZFS Snapshot
Allocate-on-demand
Allocate-on-demand
zpool

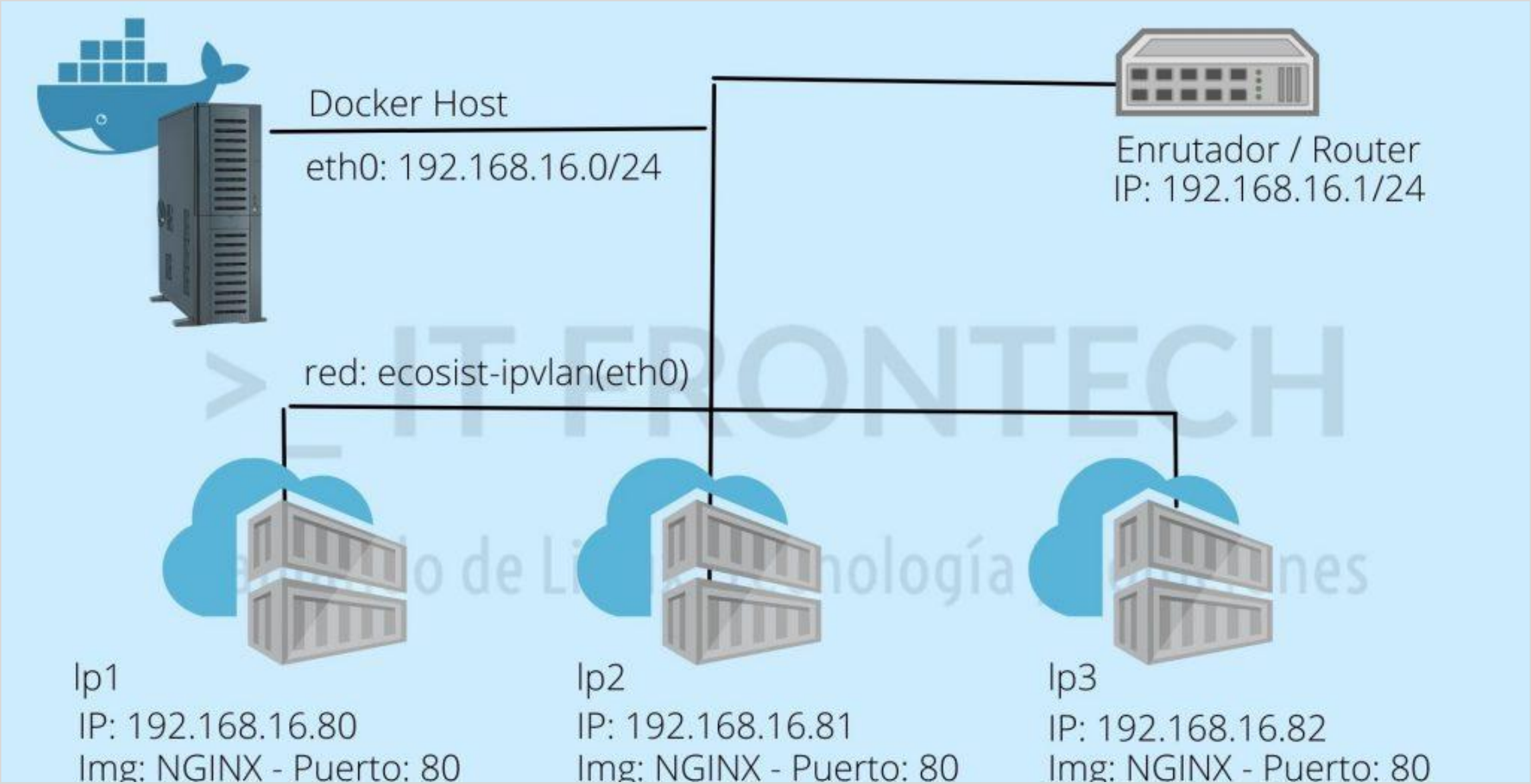

Random UUID
(All writes and deletes go in this layer)
Container layer (R/W)
91e54dfb1179
0 B
d74508fb6632
1.895 KB
c22013c84729
194.5 KB
d3a1f33e8a5a
188.1 MB
ubuntu:15.04
Cryptographic content hashes
Image layers (R/O)
Container
(based on ubuntu:15.04 image)

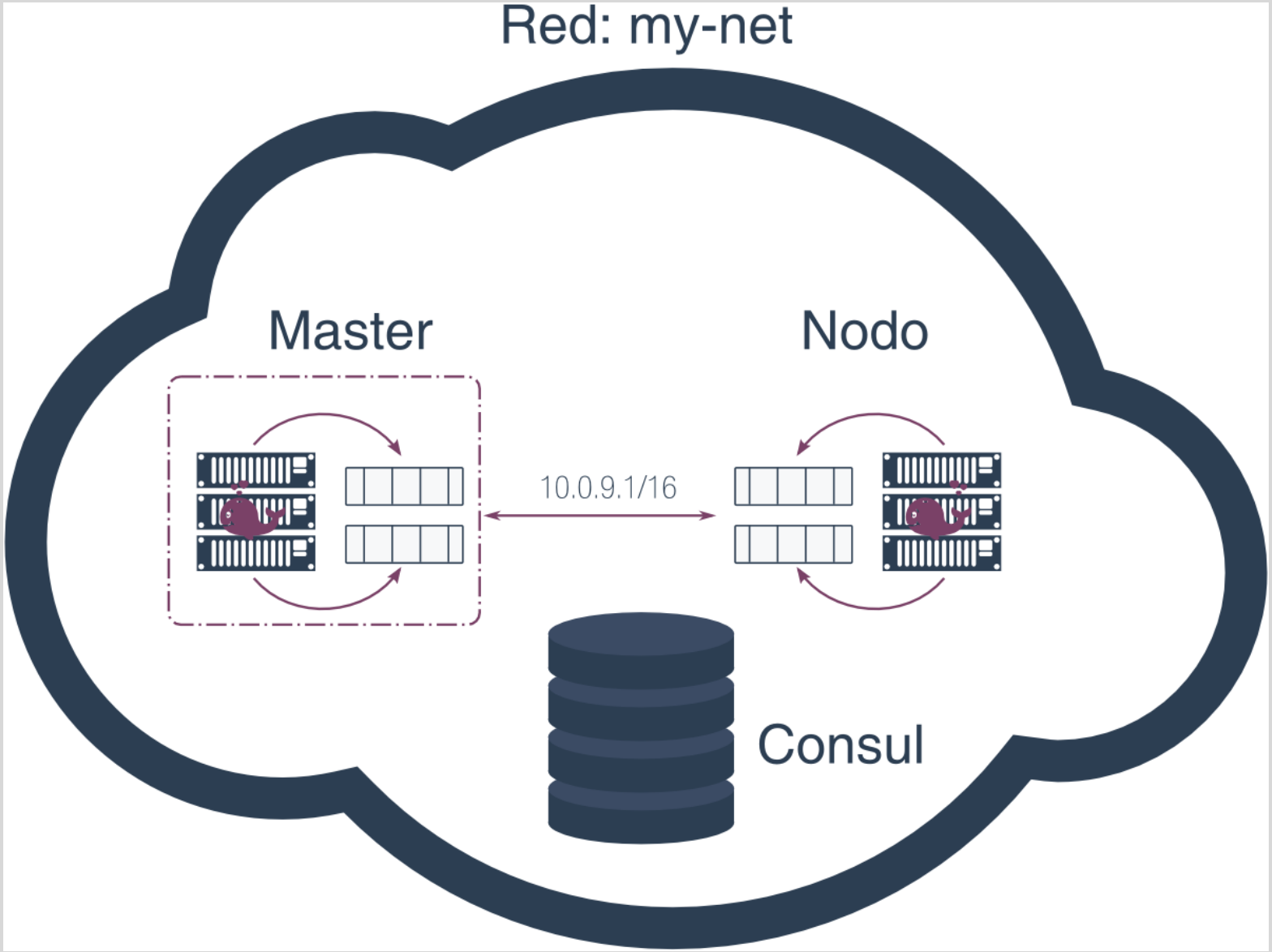

https://imagelayers.io/?images=java:latest,golang:latest,node:latest,python:latest,php:latest,ruby:latest
IMAGE LAYERS
CenturyLink
2 gb
57
33 mb
524 mb
Docker Images
Manage Images
Share URL
Copy URL
705 mb
685 mb
642 mb
517 mb
818 mb
444 mb
44 mb
19 mb
122 mb
177 mb
314 mb
98 mb
790 kb
0 bytes
0 bytes
968 kb
22 kb
0 bytes
61 bytes
0 bytes
0 bytes
0 bytes
40 mb
0 bytes
0 bytes
13 kb
0 bytes
97 mb
0 bytes
17 kb

LABORATORIO 3

LABORATORIO 4


Container 1
eth0: 172.19.0.2
Bridge
Container 2
eth0: 172.19.0.3

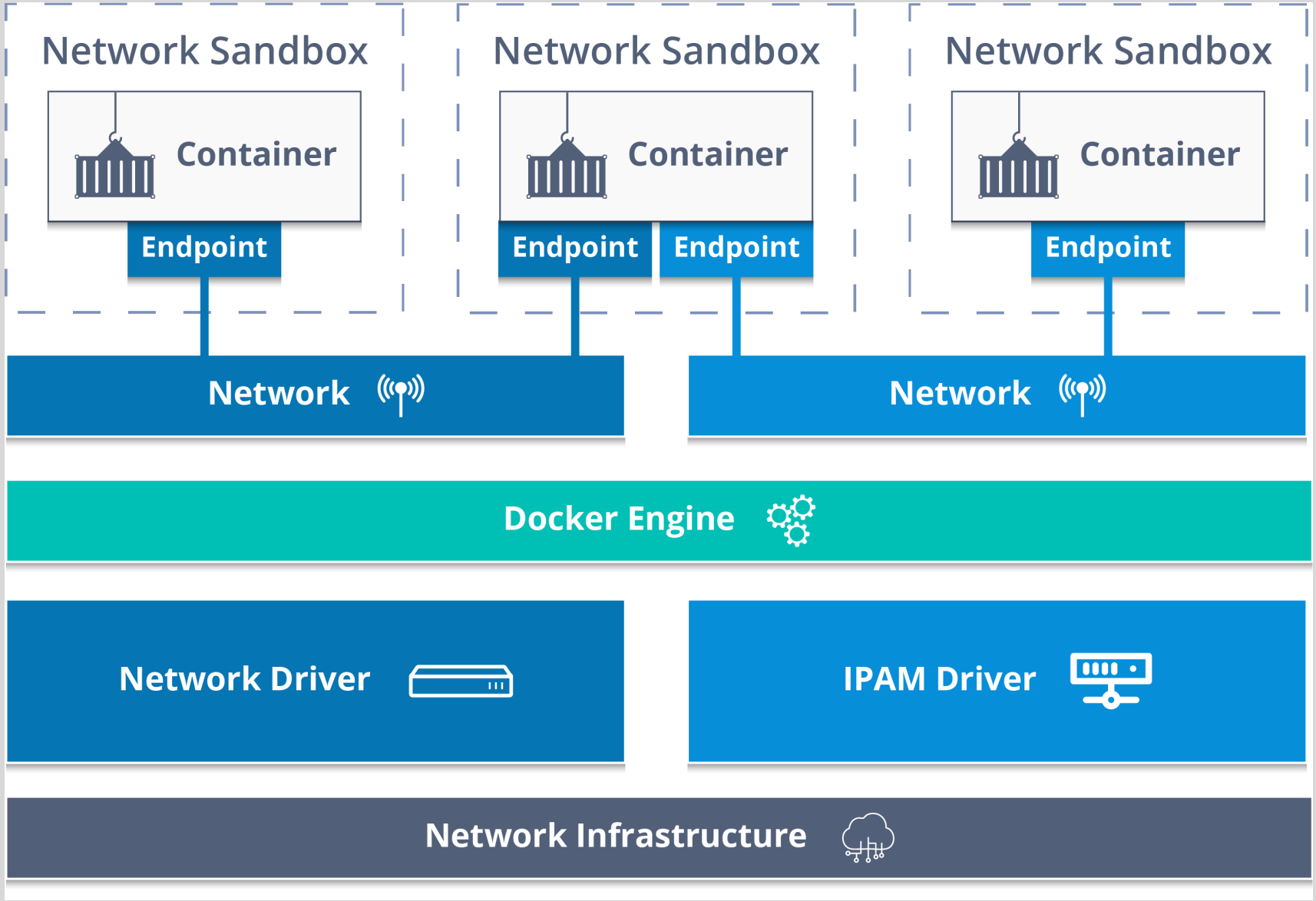

Docker Host
eth0: 192.168.16.0/24
Enrutador / Router
IP: 192.168.16.1/24
red: ecosist-ipvlan(eth0)
lp1
IP: 192.168.16.80
Img: NGINX - Puerto: 80
lp2
IP: 192.168.16.81
Img: NGINX - Puerto: 80
lp3
IP: 192.168.16.82
Img: NGINX - Puerto: 80


Red: my-net
Master
Nodo
10.0.9.1/16
Consul

LABORATORIO 5

LABS

https://training.play-with-docker.com/alacart/


Network Sandbox
Container
Endpoint
Network Sandbox
Container
Endpoint
Endpoint
Network Sandbox
Container
Endpoint
Network
Network
Docker Engine
Network Driver
IPAM Driver
Network Infrastructure

El comando principal para gestionar redes en Docker. Permite:

- Crear redes (`docker network create`).
- Listar redes existentes (`docker network ls`).
- Inspeccionar redes (`docker network inspect`).
- Conectar/desconectar contenedores a/de redes (`docker network connect` y `docker network disconnect`).
- Eliminar redes (`docker network rm`).

DOCKER NETWORK

Bridge: Red local predefinida que permite la comunicación entre contenedores en un mismo host.

Host: Utiliza la pila de red del host directamente.

None: Sin configuración de red.

REDES PREDETERMINADAS

El comando `docker network inspect <network>` proporciona detalles como:

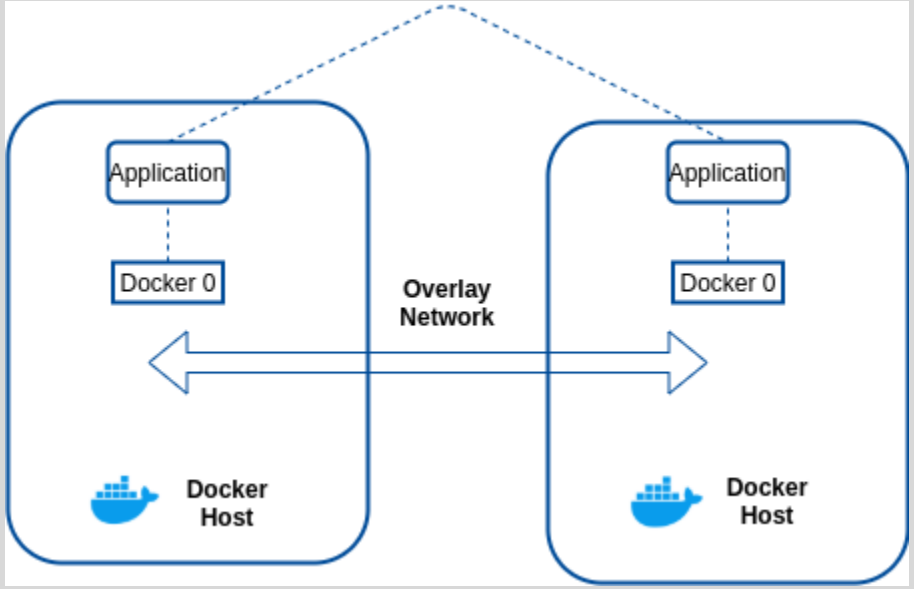
- ID de la red
- Driver utilizado
- Configuración de IP (subred y puerta de enlace)
- Contenedores conectados

Las redes bridge funcionan como un "switch virtual".

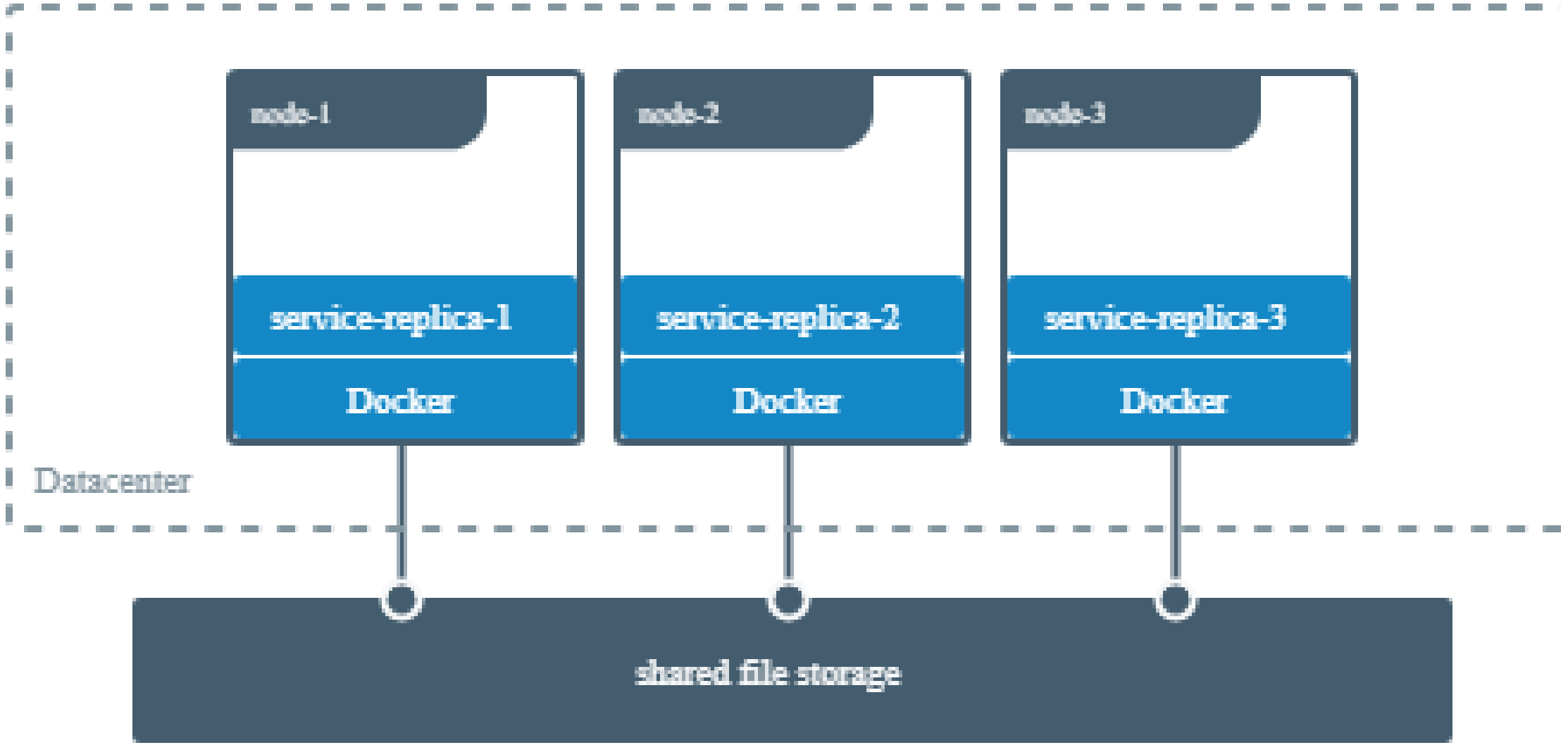
Permiten la comunicación entre contenedores en el mismo host.

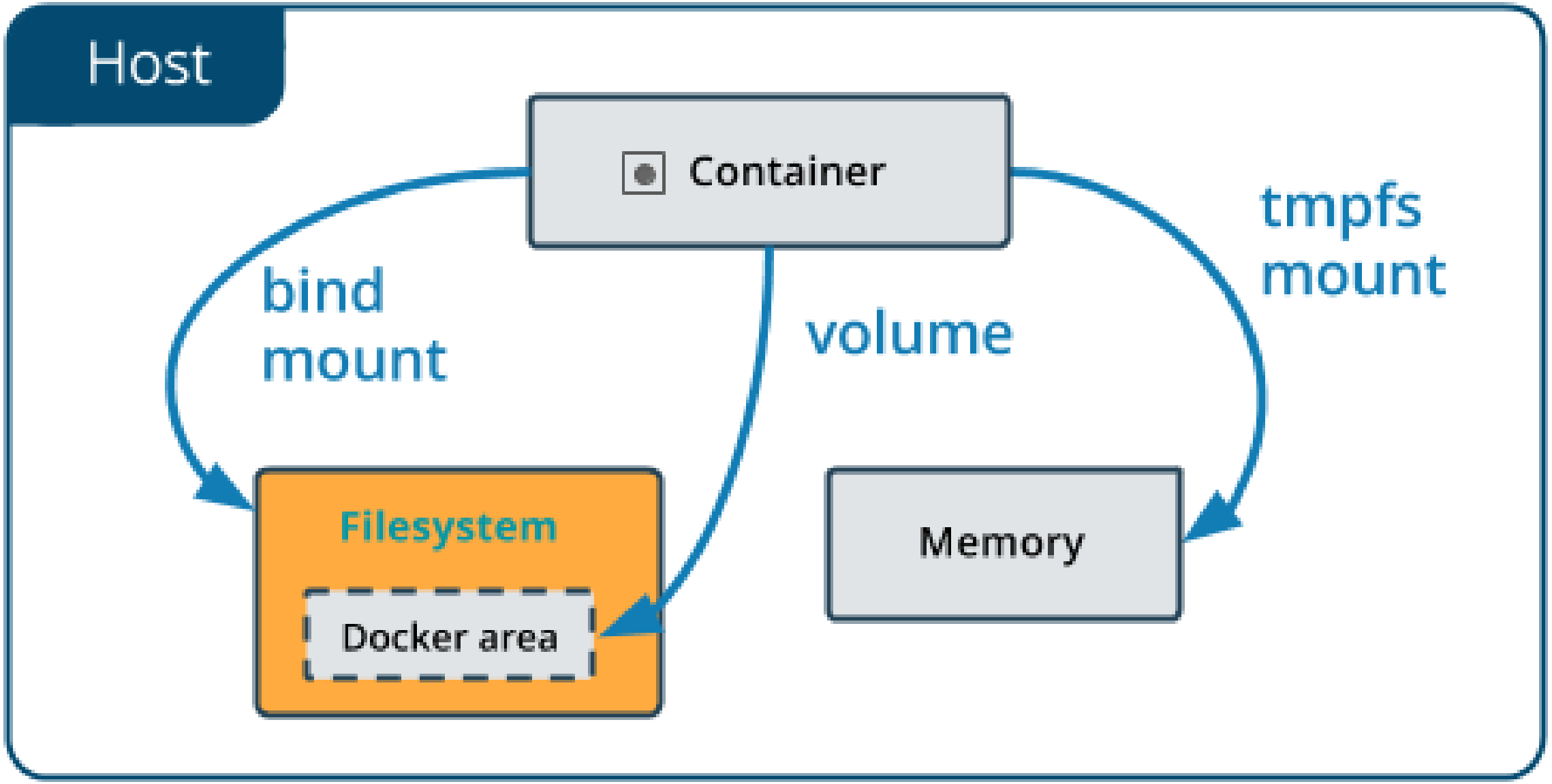
La red `bridge` es la predeterminada en nuevas instalaciones de Docker.

BRIDGE


Application
Docker 0
Overlay
Network
Application
Docker 0
Docker
Host
Docker
Host

LABORATORIO


node-1
service-replica-1
Docker
node-2
service-replica-2
Docker
node-3
service-replica-3
Docker
Datacenter
shared file storage


Host
Container
bind
mount
volume
tmpfs
mount
Filesystem
Docker area
Memory

Los contenedores utilizan capas de lectura-escritura para almacenar datos temporalmente. Estas capas se eliminan cuando el contenedor es eliminado.

- Los volúmenes almacenan datos fuera del sistema de archivos de unión (Union Filesystem).
- Los datos persisten independientemente del ciclo de vida del contenedor.

Ejemplo práctico:

1. Crear un Dockerfile:

   dockerfile Copiar código

   ```dockerfile
   FROM alpine
   VOLUME ["/data"]
   ENTRYPOINT ["/bin/sh"]
   ```

   Esto indica que la ruta `/data` debe usar un volumen.

El flag `-v` permite crear y montar volúmenes dinámicamente al ejecutar un contenedor.

Ejemplo práctico:

1. Crear un contenedor con un volumen:

   bash Copiar código

   ```bash
   docker container run --name c3 -d -v /data alpine sh -c 'ping 8.8.8.8
   ```

   Este contenedor guarda el resultado de un comando `ping` en el volumen.

2. Inspeccionar el contenedor para encontrar la ruta del volumen:

Los bind mounts conectan directamente una carpeta o archivo del host con un contenedor. Esto permite una mayor flexibilidad, especialmente en entornos de desarrollo.

Ejemplo práctico:

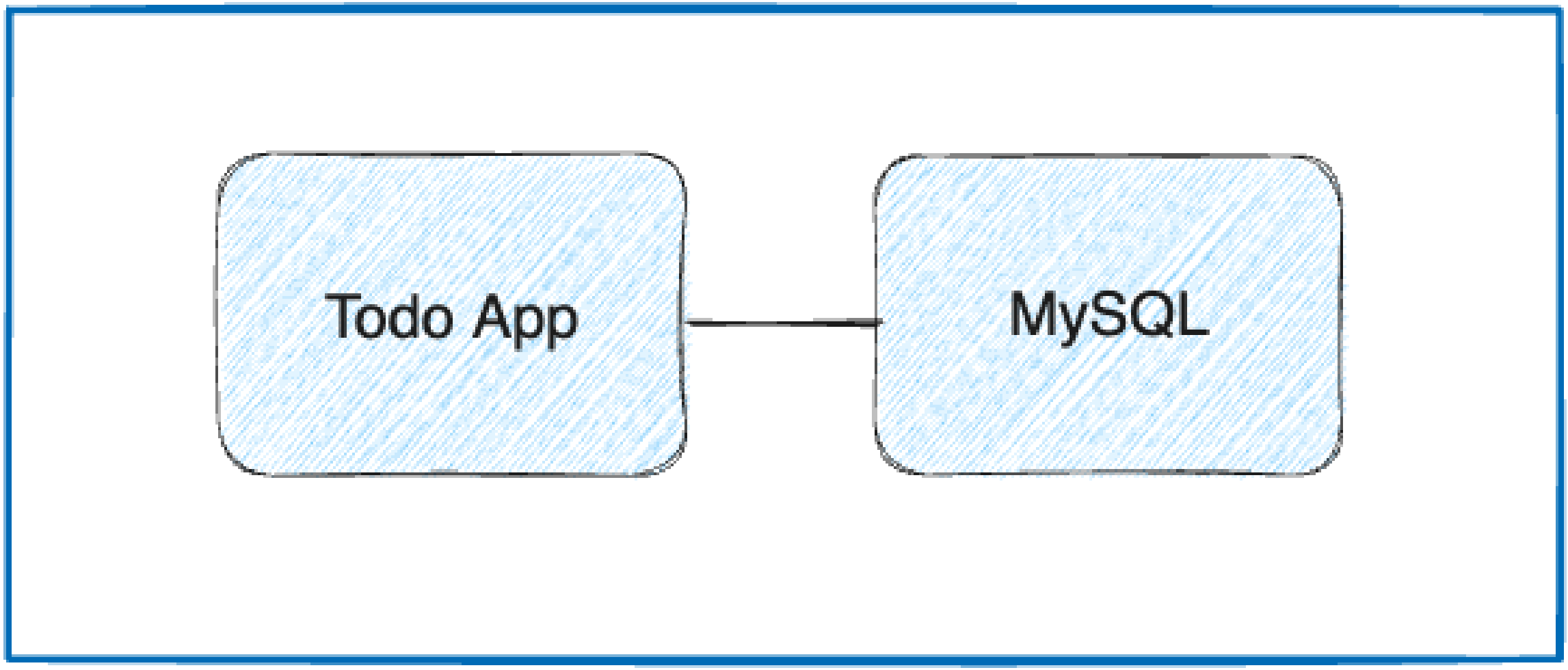
1. Montar una carpeta del host en un contenedor:

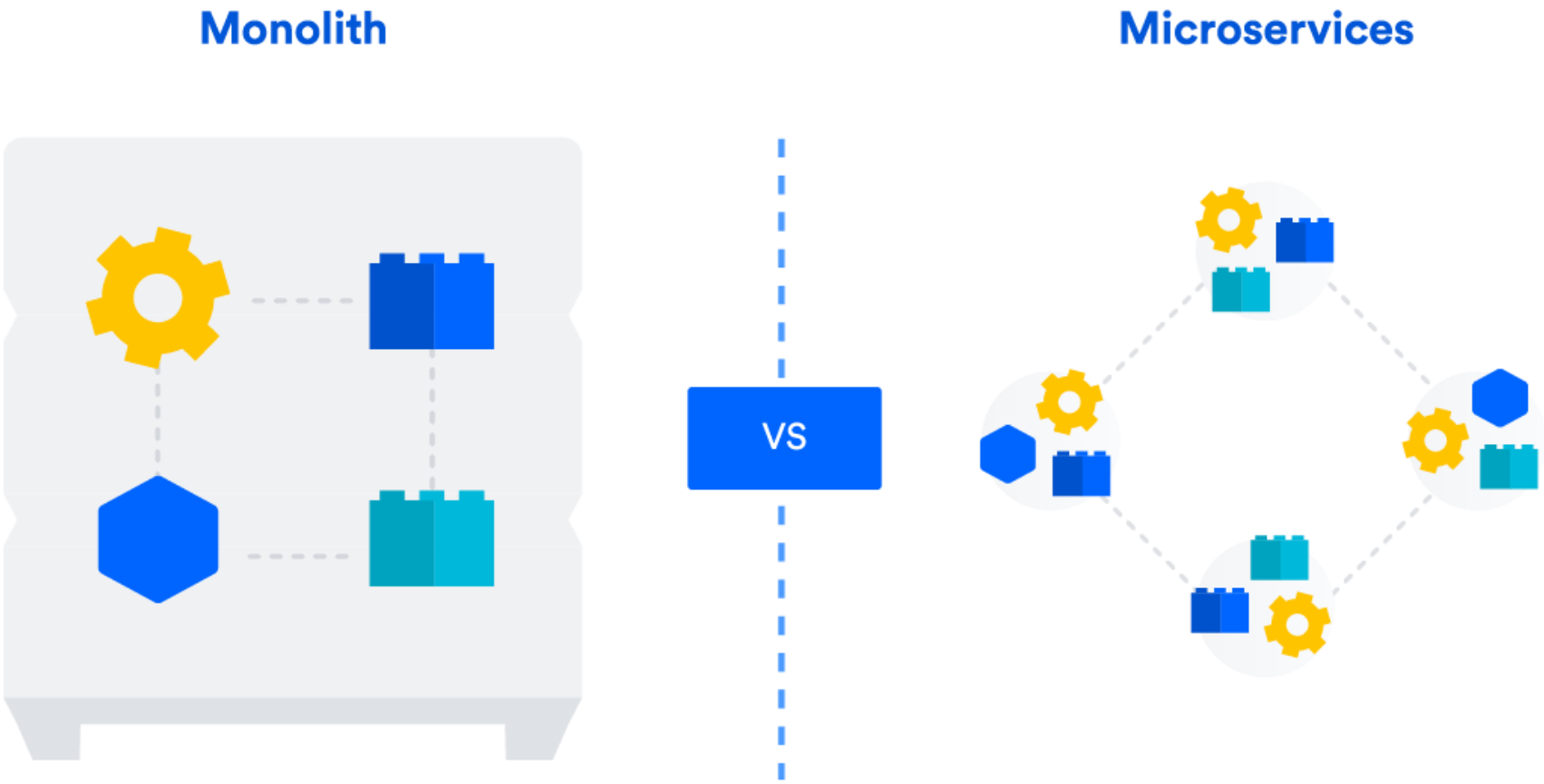
   bash Copiar código

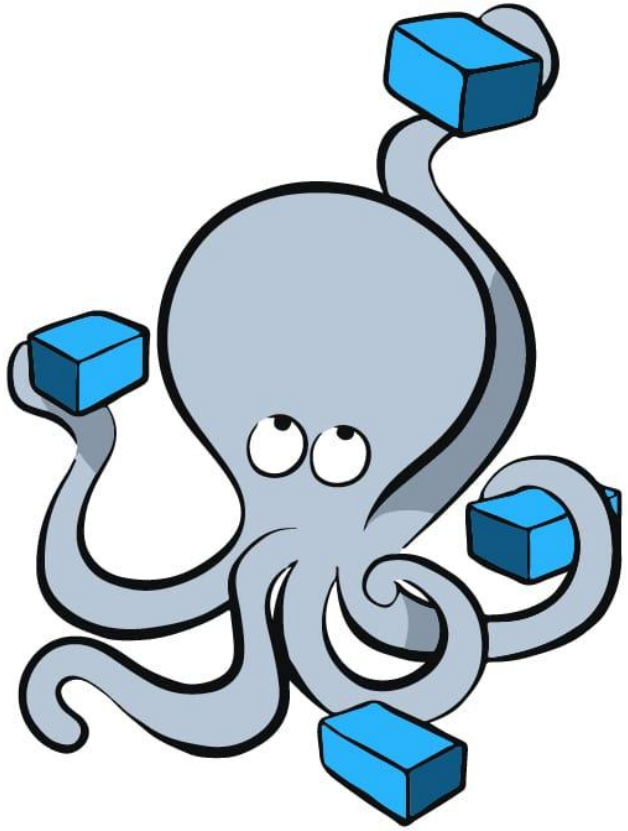
   ```bash
   docker container run -ti -v /tmp:/data alpine sh
   ```

   - El contenido de `/tmp` en el host aparece en `/data` dentro del contenedor.

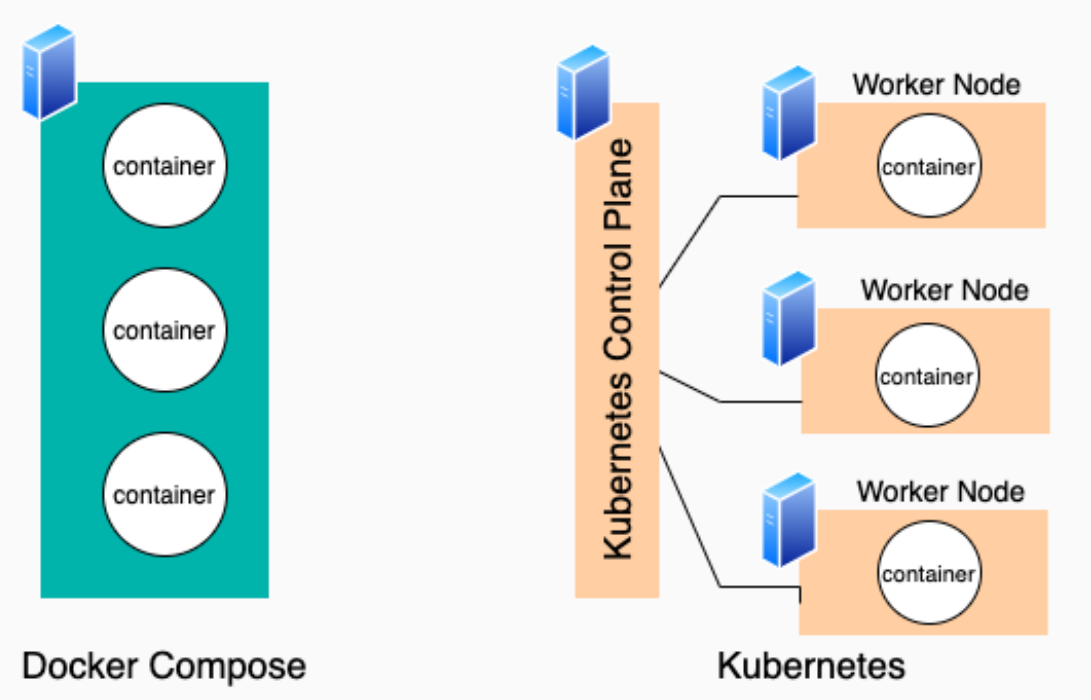
LABORATORIO

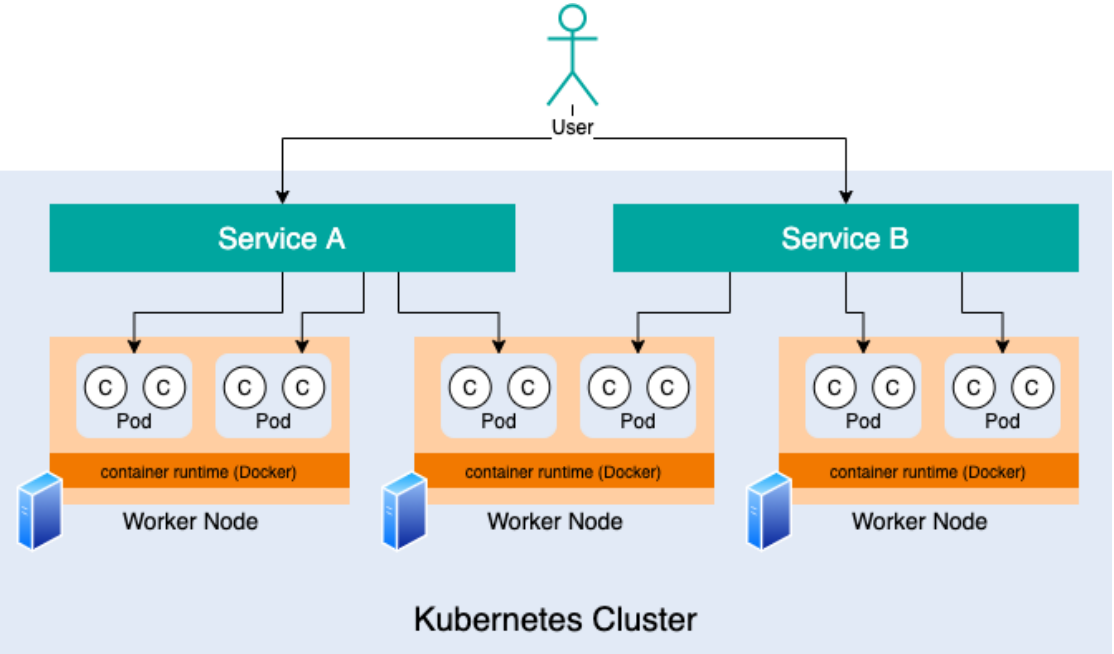

Todo App
MySQL

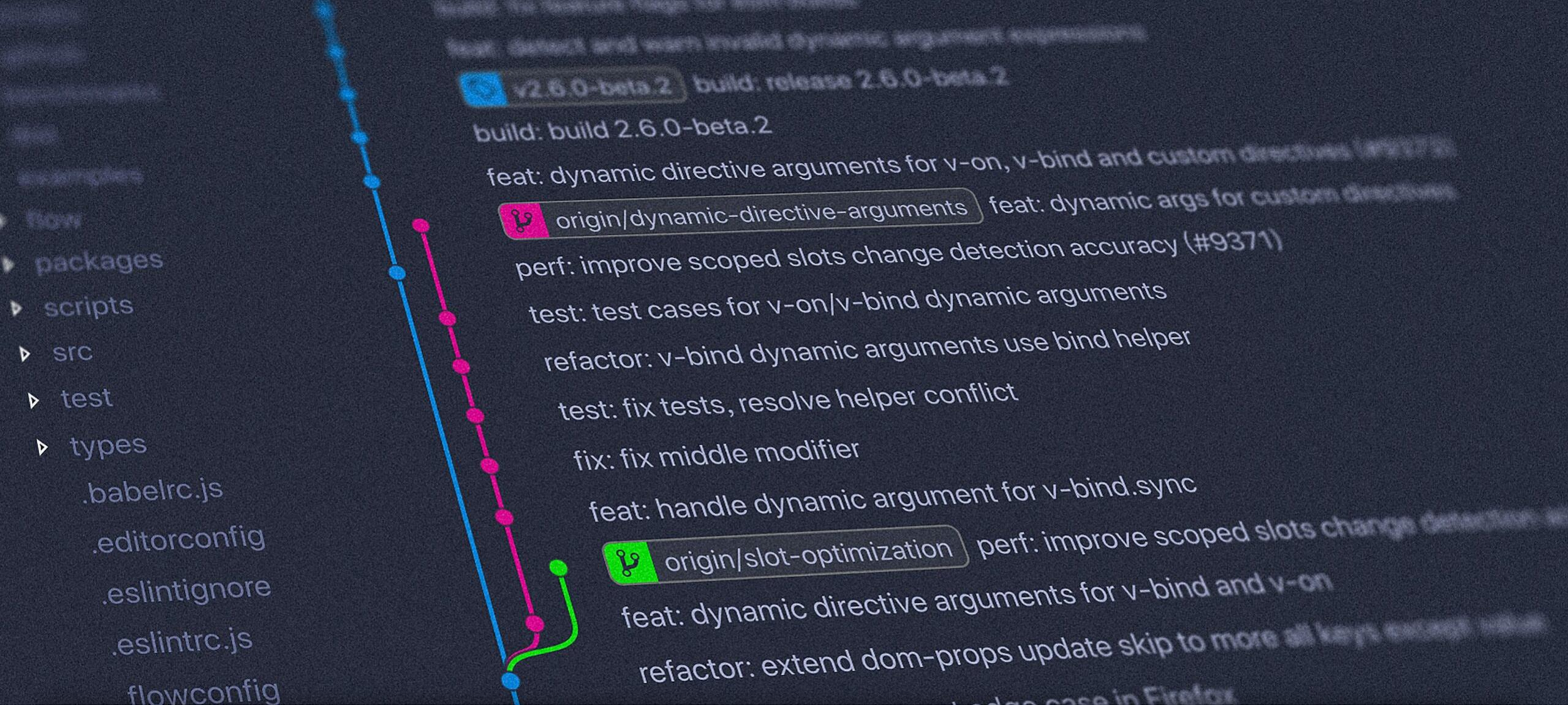

Monolith
Microservices
VS



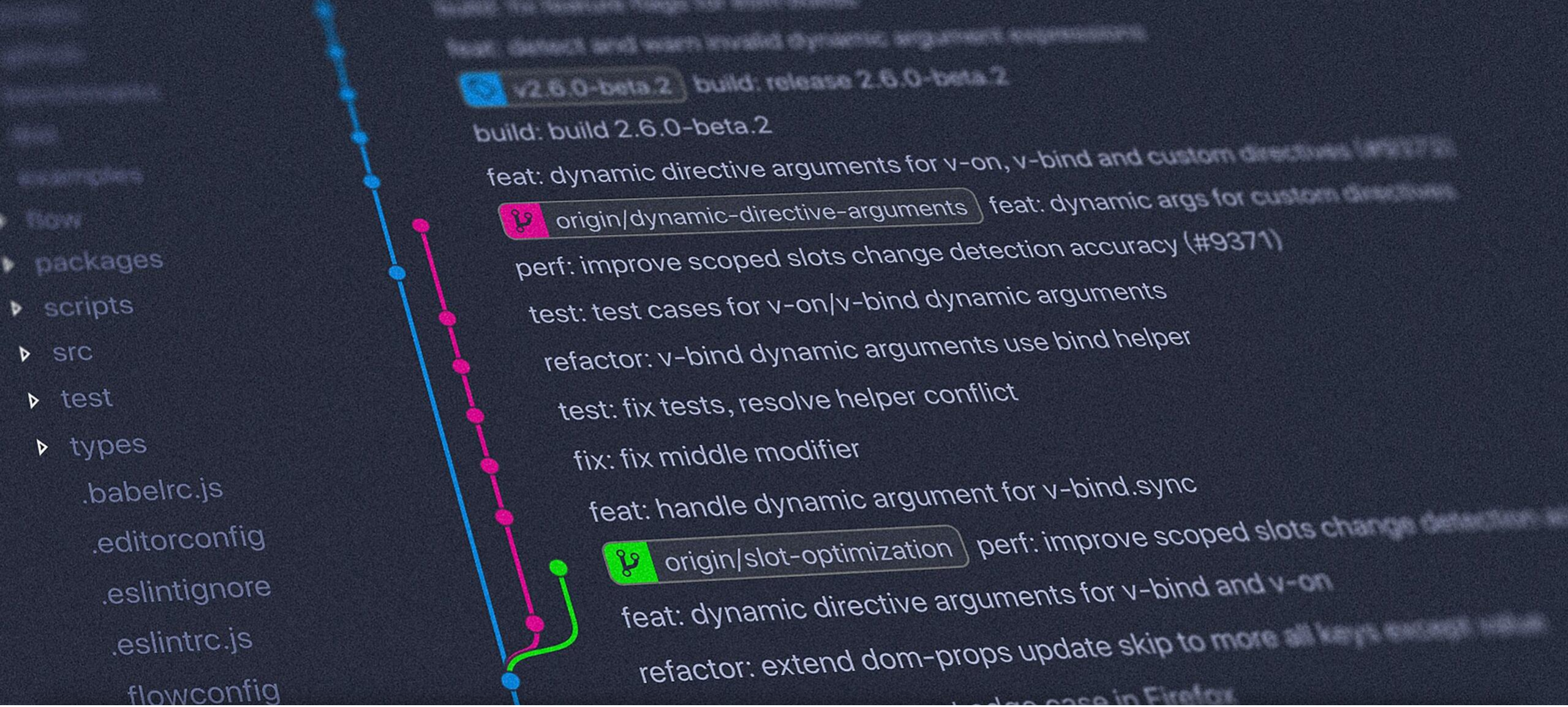
docker
Compose


container
container
container
Docker Compose
Kubernetes Control Plane
Worker Node
container
Worker Node
container
Worker Node
container
Kubernetes

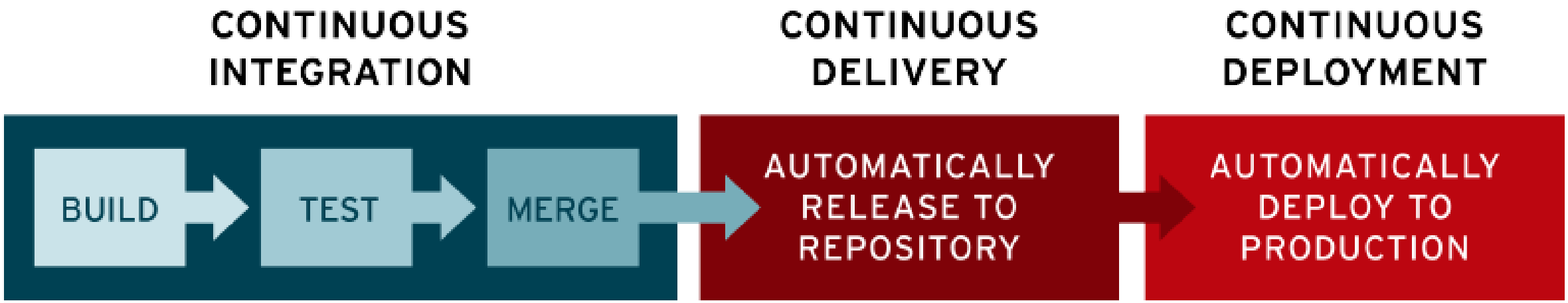

User
Service A
Service B
C C
Pod
C C
Pod
C C
Pod
C C
Pod
C C
Pod
C C
Pod
container runtime (Docker)
container runtime (Docker)
container runtime (Docker)
Worker Node
Worker Node
Worker Node
Kubernetes Cluster



LABORATORIO 6

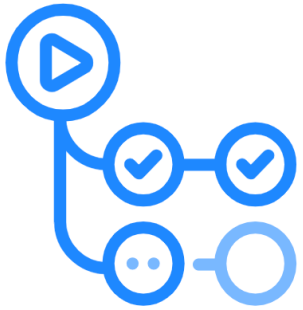


LABORATORIO 7

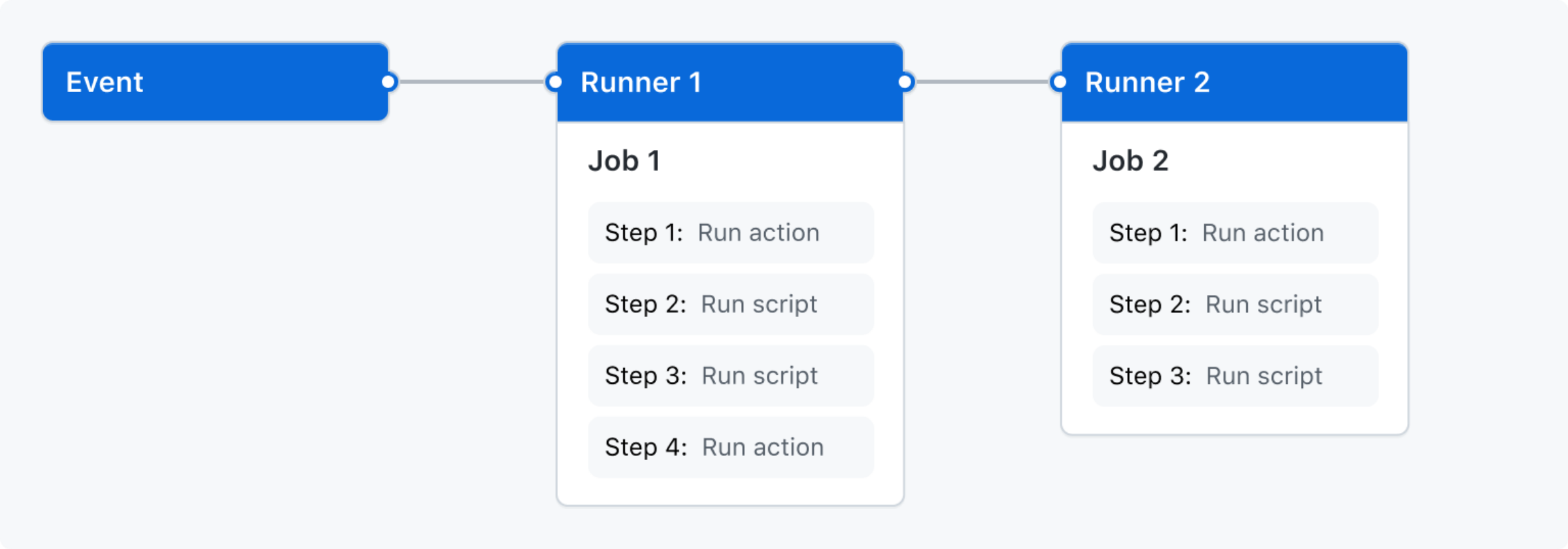

CONTINUOUS INTEGRATION
CONTINUOUS DELIVERY
CONTINUOUS DEPLOYMENT
BUILD
TEST
MERGE
AUTOMATICALLY RELEASE TO REPOSITORY
AUTOMATICALLY DEPLOY TO PRODUCTION

CI/CD

GITHUB ACTIONS



GitHub Actions

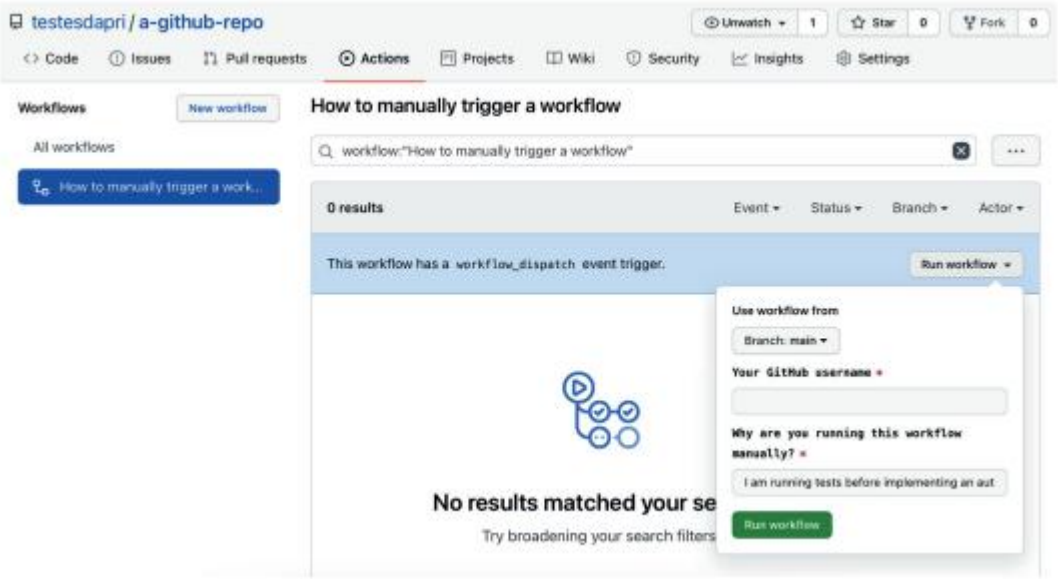


LOS COMPONENTES DE GITHUB ACTIONS

SINTAXIS

```yaml
name: Nombre-del-Workflow

on:
  evento:
    - evento-trigger

jobs:
  nombre-del-trabajo:
    runs-on: tipo-de-maquina
    steps:
      - nombre-del-paso:
          run: comando-a-ejecutar
```

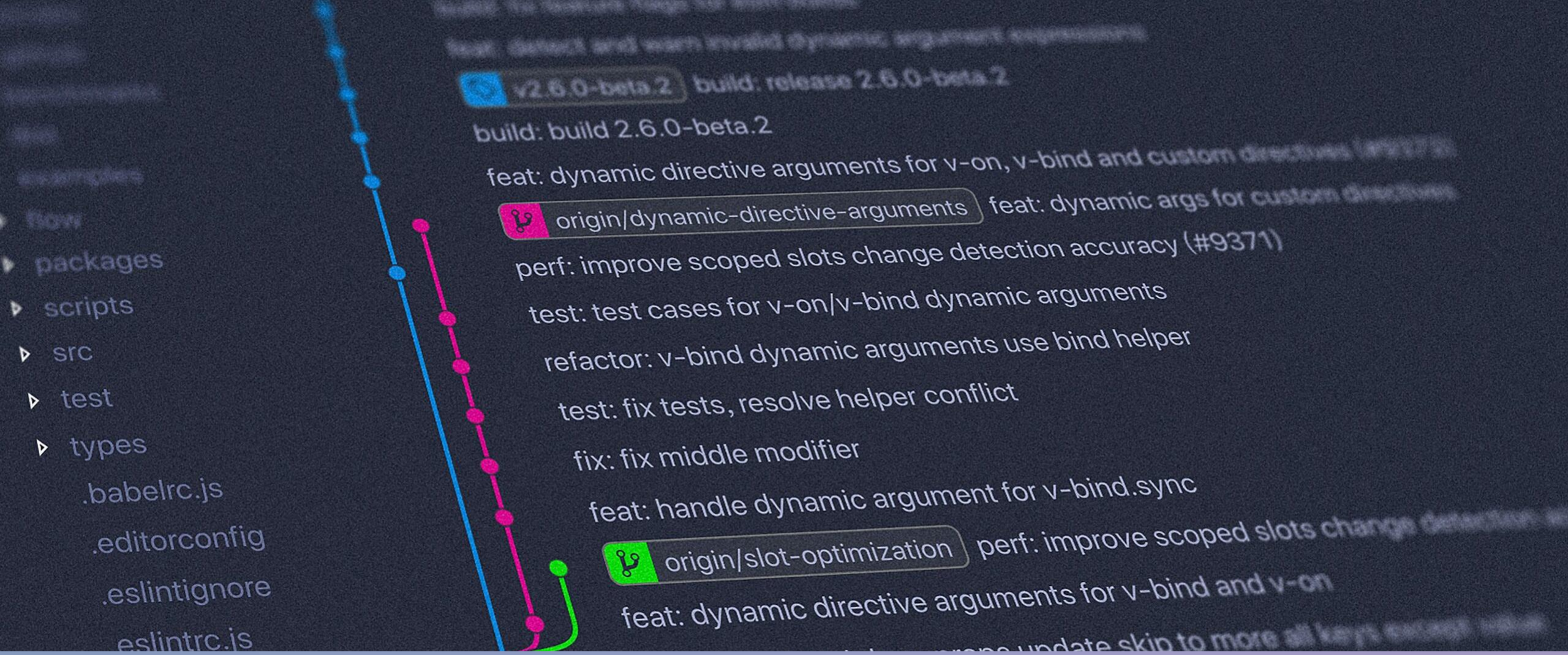
EVENTOS


testesdapri / a-github-repo
Unwatch 1
Star 0
Fork 0
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
Workflows
New workflow
All workflows
How to manually trigger a work...
How to manually trigger a workflow
workflow:"How to manually trigger a workflow"
0 results
Event
Status
Branch
Actor
This workflow has a workflow_dispatch event trigger.
Run workflow
Use workflow from
Branch: main
Your GitHub username *
Why are you running this workflow manually? *
I am running tests before implementing an aut
Run workflow
No results matched your se
Try broadening your search filters

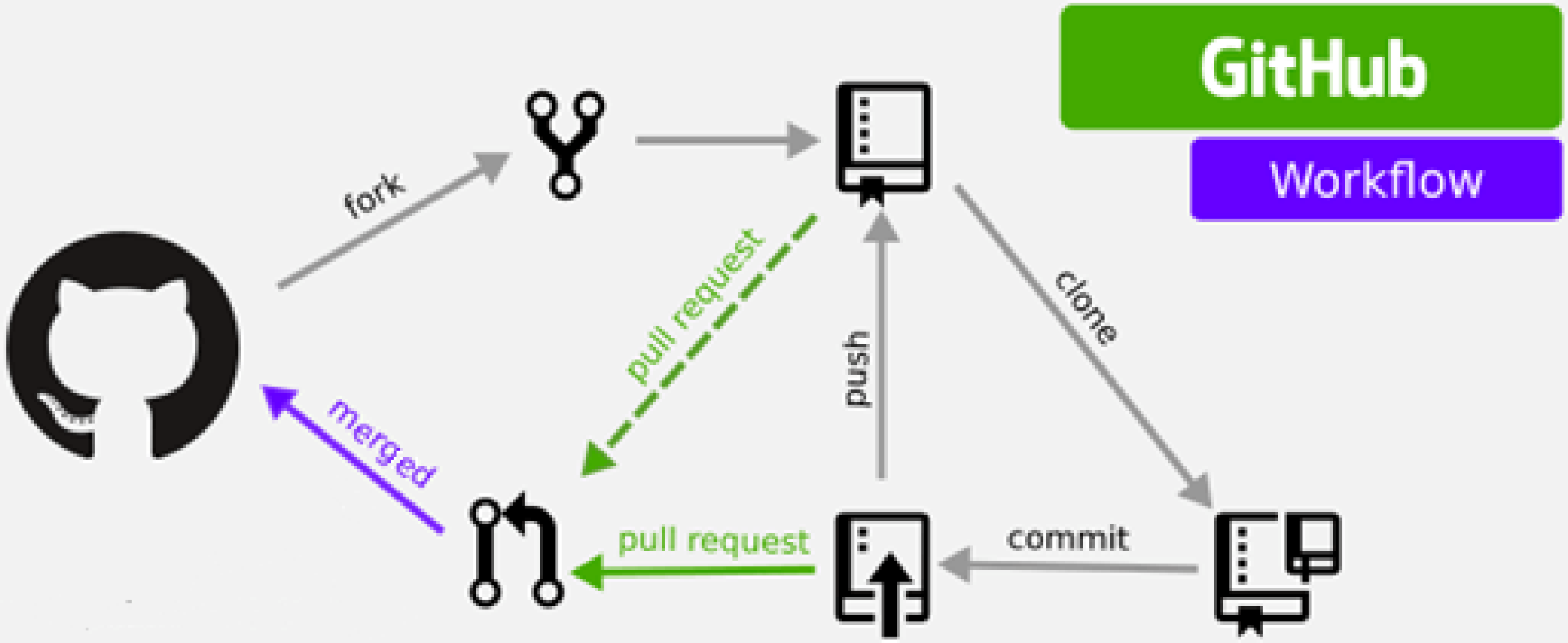
YAML

```yaml
name: Greeting on variable day

on:
  workflow_dispatch

env:
  DAY_OF_WEEK: Monday

jobs:
  greeting_job:
    runs-on: ubuntu-latest
    env:
      Greeting: Hello
    steps:
      - name: "Say Hello Mona it's Monday"
        run: echo "$Greeting $First_Name. Today is $DAY_OF_WEEK!"
        env:
          First_Name: Mona
```

AGENTES


v2.6.0-beta.2
build: release 2.6.0-beta.2
build: build 2.6.0-beta.2
feat: dynamic directive arguments for v-on, v-bind and custom
origin/dynamic-directive-arguments
feat: dynamic args for custom
perf: improve scoped slots change detection accuracy (#9371)
test: test cases for v-on/v-bind dynamic arguments
refactor: v-bind dynamic arguments use bind helper
test: fix tests, resolve helper conflict
fix: fix middle modifier
feat: handle dynamic argument for v-bind.sync
origin/slot-optimization
perf: improve scoped slots
feat: dynamic directive arguments for v-bind and v-on
flow
packages
scripts
src
test
types
.babelrc.js
.editorconfig
.eslintignore
.eslintrc.js

DEMO


GitHub
Workflow
fork
pull request
push
clone
merged
pull request
commit

Options

Manage access

Security & analysis

Branches

Webhooks

Actions

Secrets

Secrets / New secret

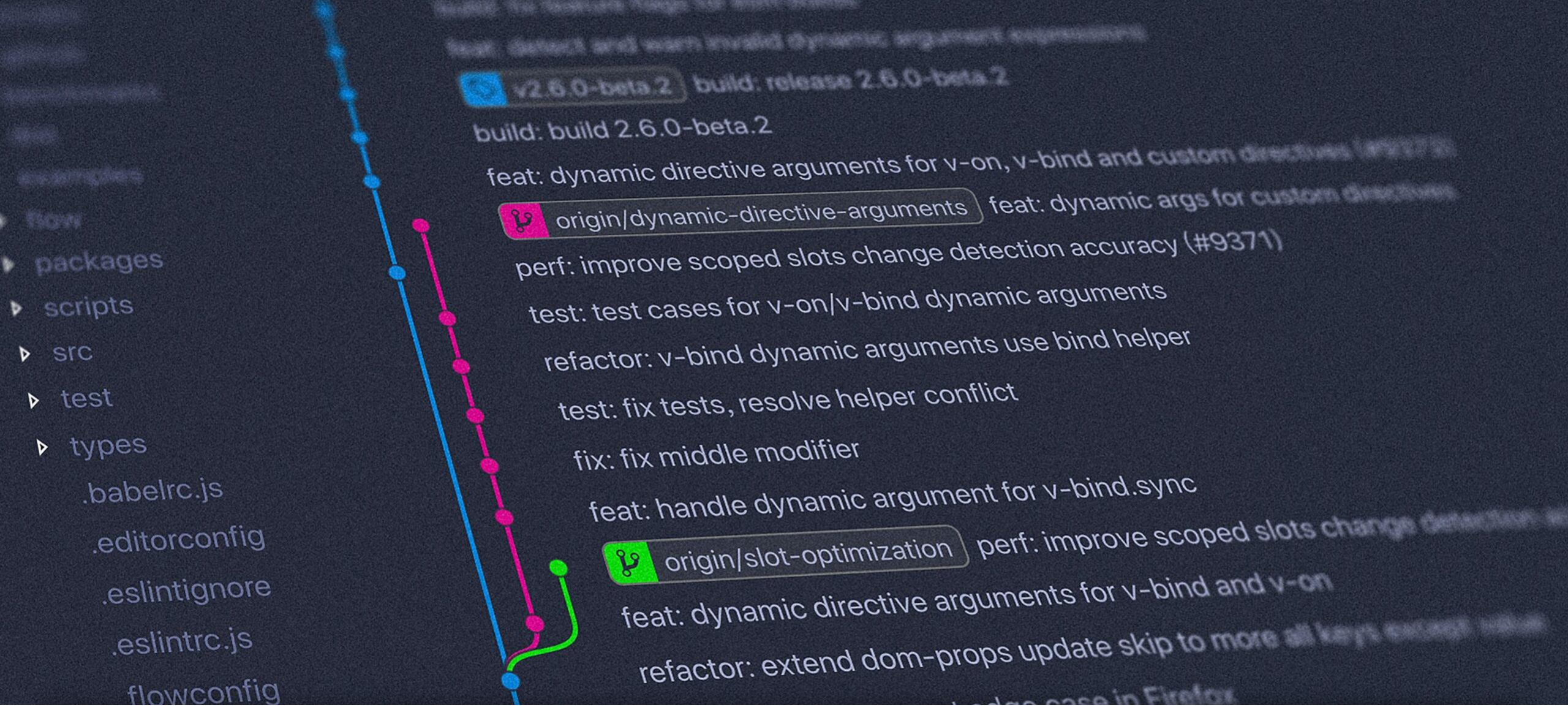
Name

SECRET_TOKEN

Value

TrustNo1

Add secret



LABORATORIO 8